

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA BAHIA

PEDRO VICTOR HIPÓLITO CABRAL

CENTRO DE AJUDA: SISTEMA INTELIGENTE DE REDIRECIONAMENTO DE COMUNICAÇÃO EM SITUAÇÕES EMERGENCIAIS

SANTO ANTÔNIO DE JESUS - BA

2026

PEDRO VICTOR HIPÓLITO CABRAL

CENTRO DE AJUDA: SISTEMA INTELIGENTE DE REDIRECIONAMENTO DE COMUNICAÇÃO EM SITUAÇÕES EMERGENCIAIS

Trabalho de Conclusão de Curso apresentado a Instituto Federal de Educação, Ciência e Tecnologia da Bahia, campus Santo Antônio de Jesus - BA, como requisito parcial para obtenção do grau de Tecnólogo em Análise e Desenvolvimento de Sistemas.

Prof. Leandro Costa Souza

SANTO ANTÔNIO DE JESUS - BA

2026

Ficha catalográfica.

Cabral, Pedro Victor Hipólito

Centro de Ajuda: Sistema Inteligente de Redirecionamento de Comunicação em Situações Emergenciais / Pedro Victor Hipólito Cabral. — Santo Antônio de Jesus, Bahia, 2026.

[TOTAL DE PÁGINAS] p.: il.

Orientador: Leandro Costa Souza.

Trabalho de Conclusão de Curso (Tecnologia em Análise e Desenvolvimento de Sistemas) — Instituto Federal de Educação, Ciência e Tecnologia da Bahia, campus Santo Antônio de Jesus, 2026.

1. Inteligência artificial.
2. Serviços de emergência.
3. Processamento multimodal.
4. Modelos de linguagem.

I. Souza, Leandro Costa. II. Instituto Federal de Educação, Ciência e Tecnologia da Bahia, campus Santo Antônio de Jesus. III. Título.

CDD: [A DEFINIR]

PEDRO VICTOR HIPÓLITO CABRAL

CENTRO DE AJUDA: SISTEMA INTELIGENTE DE REDIRECIONAMENTO DE COMUNICAÇÃO EM SITUAÇÕES EMERGENCIAIS

A banca examinadora, abaixo listada, aprova o Trabalho de Conclusão de Curso CENTRO DE AJUDA: SISTEMA INTELIGENTE DE REDIRECIONAMENTO DE COMUNICAÇÃO EM SITUAÇÕES EMERGENCIAIS elaborado por PEDRO VICTOR HIPÓLITO CABRAL como requisito parcial para obtenção do grau de Tecnólogo em Análise e Desenvolvimento de Sistemas, pelo Instituto Federal de Educação, Ciência e Tecnologia da Bahia.

Santo Antônio de Jesus - BA, sexta-feira, 18 de setembro de 2026

Comissão Examinadora

Prof. Mst. Leandro Costa Souza (IFBA)
Orientador

Prof. Dr. (IFBA)

Prof. Dr. (IFBA)

RESUMO

Este trabalho apresenta o Centro de Ajuda, uma prova de conceito destinada a facilitar a identificação dos serviços adequados para o atendimento de situações emergenciais. A proposta parte da fragmentação dos números de emergência no Brasil e das dificuldades que essa organização pode gerar, especialmente em momentos de medo, estresse ou urgência. O objetivo é permitir que o usuário descreva uma ocorrência e receba uma indicação de atendimento compatível com sua necessidade. Para isso, foi desenvolvida uma solução organizada em aplicação cliente e servidor, com inteligência artificial para interpretar relatos em texto, áudio e imagem, considerando informações de localização e horário. A avaliação comparou sete modelos em quatro cenários, com uma execução por combinação de modelo e cenário, observando compatibilidade com as entradas e adequação das respostas. Nas condições testadas, apenas o Gemini 3.5 Flash processou diretamente todas as modalidades nos quatro cenários. Os demais apresentaram limitações que motivaram uma análise complementar da utilização conjunta de modelos especializados. Os resultados indicam potencial técnico para apoiar o direcionamento de solicitações, mas a ausência de repetições limita sua generalização. Conclui-se que a proposta oferece uma base para novos estudos e aperfeiçoamentos, dependendo de avaliações mais amplas, validação dos contatos e melhorias de confiabilidade antes de uma eventual utilização em situações reais, sem substituir as centrais oficiais.

Palavras-chave: Inteligência Artificial. Emergência. Multimodal. LLM.

ABSTRACT

This study presents Centro de Ajuda, a proof of concept designed to help users identify appropriate services for emergency situations. The proposal addresses the fragmentation of emergency telephone numbers in Brazil and the difficulties this arrangement may create, particularly in moments of fear, stress, or urgency. Its objective is to allow users to describe an incident and receive a service recommendation suited to their needs. To this end, a solution comprising a client application and a server was developed, using artificial intelligence to interpret text, audio, and image reports alongside location and time information. The evaluation compared seven models across four scenarios, with one execution per model–scenario combination, examining input compatibility and response appropriateness. Under the tested conditions, only Gemini 3.5 Flash directly processed all input modalities across the four scenarios. The remaining models showed limitations that motivated a complementary analysis of combining specialized models. The findings suggest technical potential for supporting emergency service selection, although the lack of repeated trials limits their generalizability. The study concludes that the proposal provides a foundation for further research and development, requiring broader evaluation, verification of contact information, and improvements in reliability before potential use in real situations, without replacing official emergency dispatch centers.

Keywords: Artificial Intelligence. Emergency. Multimodal. LLM.

LISTA DE FIGURAS

Figura 1 – Fluxo de interação entre usuário, servidor e inteligência artificial 16

Figura 2 – Fluxo da API Fetch com Promise 43

Figura 3 – Fluxo de comunicação entre frontend, backend e APIs externas 47

Figura 4 – Arquitetura da aplicação cliente 50

Figura 5 – Tela inicial do Centro de Ajuda 53

Figura 6 – Tela de espera 62

Figura 7 – Tela de resposta com recomendação 64

Figura 8 – Arquitetura geral do sistema 76

LISTA DE TABELAS

Tabela 1 – Comparação de modelos quanto aos critérios técnicos 33

Tabela 2 – Cenários testados 89

Tabela 3 – Exemplo de métricas por modelo 90

Tabela 4 – Comparação por critérios — modelo isolado 92

Tabela 5 – Viabilidade por ecossistema (composição sequencial) 94

LISTA DE CÓDIGOS

Código 1 – Campo de texto da ocorrência 54
Código 2 – Captura de texto via DOM 54
Código 3 – Solicitação de acesso ao microfone 55
Código 4 – Gravação de áudio com MediaRecorder 56
Código 5 – Captura de imagem 58
Código 6 – Envio da ocorrência ao servidor 60
Código 7 – Tratamento da resposta do servidor 66
Código 8 – Redirecionamento para ligação 67
Código 9 – Inicialização do servidor 68
Código 10 – Configuração do Express 69
Código 11 – Tratamento das solicitações ao servidor 71
Código 12 – Geocodificação reversa 73
Código 13 – Conversão de arquivos para Base64 75
Código 14 – System Prompt 78
Código 15 – Assistant Prompt contextual 79
Código 16 – Construção do conteúdo multimodal 80
Código 17 – Envio da requisição ao OpenRouter 84
Código 18 – Registro da resposta e das métricas da requisição 85
Código 19 – Validação da estrutura da resposta 87

LISTA DE ABREVIATURAS E SIGLAS

API	Application Programming Interface
APIS	Application Programming Interfaces
ASR	Automatic Speech Recognition
CNNS	Convolutional Neural Networks
CORS	Cross-Origin Resource Sharing
DOM	Document Object Model
IA	Inteligência Artificial
IAS	Inteligências Artificiais
LLMS	Large Language Models
PRF	Policial Rodoviário Federal
PWA	Progressive Web App
RNNS	Recurrent Neural Networks
SAMU	Serviço de Atendimento Móvel de Urgência
URI	Uniform Resource Identifier

SUMÁRIO

1 Introdução 13

1.1 Justificativa 13

1.1.1 O que é o número de emergência universal 13

1.1.2 Números de emergência no Brasil 14

1.2 Objetivo 15

1.2.1 Objetivos Específicos 16

1.3 Correlatos 16

1.3.1 RapidSOS 17

1.3.2 911 e 112 17

2 Revisão Bibliográfica 19

2.1 Inteligência Artificial 19

2.1.1 Aprendizado de Máquina 19

2.1.2 Redes Neurais e Deep Learning 20

2.1.3 Arquitetura Transformer 22

2.2 LLMs 23

2.2.1 Engenharia de Prompt 24

2.3 Modelos Multimodais 25

2.3.1 Processamento de Imagem 26

2.3.2 Processamento de Áudio 28

2.3.3 Processamento de Vídeo 29

3 Metodologia 31

3.1 Critérios de Escolha para o(s) Modelo(s) LLM 31

3.1.1 GPT-5.5 33

3.1.2 Claude Opus 4.8 34

3.1.3 Llama 4 Maverick 35

3.1.4 Qwen 3.6 Plus 36

3.1.5 Gemini 3.5 Flash 37
3.1.6 Nemotron 3 Nano Omni 38
3.1.7 North Mini Code 39
3.2 Requisitos do Frontend 40
3.2.1 Infraestrutura Frontend 40
3.2.2 Interface do Usuário 41
3.2.3 Compatibilidade Multiplataforma 41
3.2.4 Comunicação Cliente-Servidor 42
3.3 Requisitos do Backend 43
3.3.1 Infraestrutura Backend 44
3.3.2 Ambiente de Execução 44
3.3.3 Estrutura da Aplicação 45
3.3.4 Integração com Serviços Externos 45
3.4 Fluxo de Comunicação 46
4 Desenvolvimento 48
4.1 Abordagem Geral 48
4.2 Implementação da Aplicação Cliente 48
4.2.1 Arquitetura do Cliente 49
4.2.2 Tecnologias da Aplicação Cliente 50
4.2.3 Tela Inicial 51
4.2.4 Tela de Espera 61
4.2.5 Tela de Resposta 63
4.3 Implementação do Servidor 67
4.4 Construção do Prompt 76
4.4.1 Contextualização da Ocorrência 81
4.4.2 Formato de Saída (JSON) 82
4.5 Integração com Modelos de Linguagem 83

4.5.1 Comunicação com o OpenRouter 83
5 Análise dos Resultados 88
5.1 Modelos Testados 88
5.2 Cenários Testados 89
5.3 Coleta de Resultados 90
5.4 Comparação por Critérios 91
5.4.1 Segunda Análise 92
5.5 Análise Individual dos Modelos 94
5.5.1 GPT-5.5 95
5.5.2 Claude Opus 4.8 96
5.5.3 Llama 4 Maverick 97
5.5.4 Qwen 3.6 Plus 98
5.5.5 Gemini 3.5 Flash 99
5.5.6 Nemotron 3 Nano Omni 100
5.5.7 North Mini Code 101
5.6 Conclusão do Benchmarking 102
6 Conclusão 104
6.1 Avaliação dos Modelos 104
6.2 Propostas Futuras 105
6.2.1 Aperfeiçoamento da aplicação 105
6.2.2 Ampliação dos cenários de avaliação 106
6.2.3 Avaliação de outros modelos 106
6.2.4 Validação das recomendações 107
6.3 Considerações Finais 108
REFERÊNCIAS 109

1 Introdução

O acionamento de serviços de emergência no Brasil ainda é marcado pela descentralização dos recursos públicos e privados, causando confusão pública, falhas de acesso, dentre outros fatores que comprometem a qualidade das respostas dos órgãos competentes. Este capítulo define o que é o número de emergência universal, qual tipo de problema é causado quando o serviço não é disponibilizado para a população, e como esse problema pode ser resolvido através de um aplicativo, especialmente quando a máquina pública não disponibiliza tais ferramentas.

1.1 Justificativa

Para definir uma estratégia, primeiro é necessário entender os principais desafios que as motivam, nesse cenário, com ênfase às limitações enfrentadas pela população diante do baixo nível de preparo e acesso à informação.

1.1.1 O que é o número de emergência universal

Segundo Moss (2017), o número de emergência universal é, essencialmente, um identificador telefônico único criado para centralizar o acesso a qualquer tipo de urgência. Sua função é permitir que uma pessoa consiga, em qualquer situação crítica, acionar socorro imediato sem precisar distinguir qual órgão é responsável pelo atendimento. Em vez de vários números de emergência competindo por espaço, existe um ponto único centralizado capaz de encaminhar a chamada, através de informações atualizadas e localizadas, criando um canal simples, memorável e intuitivo. A autora descreve o número de emergência universal como um mecanismo que unifica a porta de entrada do sistema, reduzindo barreiras cognitivas, consequentemente facilitando o acionamento rápido e preciso do serviço.

De acordo com Moss (2017), diversos países adotaram modelos de números de emergência universal ao longo das décadas, estruturando sistemas nacionais centrali-

zados de emergência que operam a partir de um único canal de acesso. O Reino Unido, por exemplo, implementou o número 999 como sistema nacional de emergência, enquanto os Estados Unidos consolidaram o uso do famoso 911 para o acionamento de polícia, bombeiros e serviços médicos. No entanto, a implementação dessa ferramenta não ocorreu de forma uniforme em todos os países, existindo diferenças estruturais, tecnológicas e administrativas que impactaram a possibilidade de integração dos serviços unificados de emergência (MOSS, 2017).

1.1.2 Números de emergência no Brasil

No ambiente brasileiro, o atendimento emergencial permanece distribuído entre diversos diferentes números especializados, cada um associado a um órgão que socorre tipos específicos de ocorrência. Entre os principais exemplos estão o 190, destinado ao acionamento da Polícia Militar em situações de violência, crimes e ameaças à segurança pública; o 192, responsável pelo Serviço de Atendimento Móvel de Urgência (SAMU), voltando para emergências médicas; e o 193, destinado ao Corpo de Bombeiros para casos de incêndios, acidentes e resgates. Esse modelo fragmentado busca especializar o atendimento conforme a natureza da ocorrência, concluindo que o cidadão possui plena consciência da existência de múltiplos números de emergência (TELECOMUNICAÇÕES, 2025).

De acordo com a pesquisa de Unijuí (2024), a fragmentação dos números de emergência brasileiros ainda gera dificuldades significativas no reconhecimento e diferenciação dos serviços pela população brasileira. O estudo publicado identificou que, embora o número 190 apresente alto índice de reconhecimento correto (92.2%), há considerável confusão em relação às atribuições do SAMU (192) e do Corpo de Bombeiros (193), que apresentaram índices de reconhecimento correto de apenas 72.6% e 70.8%, respectivamente. Além disso, o estudo aponta que 23% dos entrevistados associaram incorretamente o número 192 ao Corpo de Bombeiros, enquanto 24.2% relacionaram

equivocadamente o número 193 ao SAMU. A pesquisa conclui que parte da população não consegue diferenciar adequadamente os serviços disponíveis, especialmente em situações de estresse, o que pode retardar o acionamento do socorro adequado, consequentemente agravando a ocorrência. Os autores apontam que a unificação dos núcleos de atendimento é uma forte possibilidade para reduzir dúvidas e simplificar o acesso aos serviços emergenciais.

1.2 Objetivo

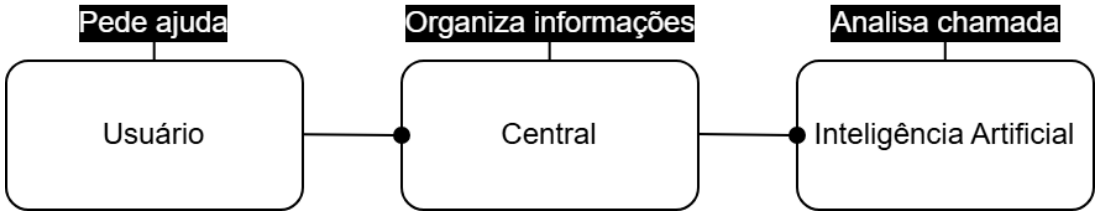
Devido à existência de múltiplos números de emergência no Brasil e às dificuldades da população em diferenciar corretamente os serviços responsáveis por cada tipo de ocorrência, o presente trabalho tem como objetivo desenvolver uma prova de conceito de um sistema inteligente de redirecionamento de comunicação em situações emergenciais, denominado Centro de Ajuda. A proposta consiste em uma aplicação móvel, integrada a um servidor responsável pelo processamento das solicitações realizadas pelos usuários.

A solução permite que o usuário envie relatos em formato textual, sonoro ou visual descrevendo a situação enfrentada, possibilitando que uma Inteligência Artificial (IA) interprete o contexto da ocorrência e forneça informações relevantes sobre qual órgão ou serviço de emergência deve ser acionado. O sistema busca atuar como uma camada intermediária de apoio à decisão, reduzindo dúvidas relacionadas aos números emergenciais e auxiliando no direcionamento adequado da solicitação à entidade final.

A lógica do projeto atribui à inteligência artificial a responsabilidade de analisar a situação utilizando diferentes dados contextuais, como localização geográfica, data, horário e descrição da ocorrência. A partir dessas informações, o sistema pode interpretar características regionais, buscar notícias com informações relevantes, compreender o contexto da emergência e identificar quais serviços possuem maior compatibilidade com a situação relatada. Dessa forma, busca-se reduzir ambiguidades no acionamento dos serviços emergenciais e melhorar a acessibilidade ao atendimento adequado.

O fluxo geral de interação entre usuário, servidor e inteligência artificial pode ser visualizado na Figura 1.

Figura 1 – Fluxo de interação entre usuário, servidor e inteligência artificial



Elaborado pelo próprio autor (2026).

1.2.1 Objetivos Específicos

Com esse objetivo definido, este trabalho consiste em desenvolver uma aplicação móvel capaz de permitir o envio de relatos emergenciais em formato textual, sonoro e visual; implementar um servidor responsável pelo recebimento, processamento e gerenciamento das informações enviadas pelos usuários; integrar modelos de inteligência artificial para interpretação contextual das ocorrências relatadas; utilizar dados complementares, como localização geográfica, data e horário, para auxiliar na análise contextual das emergências; identificar e recomendar o órgão ou serviço de emergência mais adequado para cada situação reportada; reduzir ambiguidades relacionadas à diferenciação dos números de emergência existentes no Brasil; avaliar a viabilidade da utilização de inteligência artificial como ferramenta de apoio ao redirecionamento de solicitações emergenciais; e desenvolver uma prova de conceito funcional capaz de demonstrar o fluxo completo de comunicação entre usuário, servidor e sistema inteligente.

1.3 Correlatos

Para justificar a proposta apresentada anteriormente, torna-se necessário compreender o estado atual das tecnologias e sistemas relacionados ao atendimento emergencial. No cenário contemporâneo, é possível identificar tanto iniciativas privadas desenvolvidas com o objetivo de complementar limitações estruturais dos sistemas

públicos de emergência, quanto modelos públicos integrados criados para centralizar e otimizar o acionamento de serviços emergenciais. Essas soluções buscam reduzir o tempo de resposta, melhorar a disponibilidade de informações contextuais e aumentar a eficiência na comunicação entre cidadãos e órgãos responsáveis pelo socorro.

1.3.1 RapidSOS

Representando a iniciativa privada, o *RapidSOS* consiste em uma plataforma voltada para integração tecnológica entre dispositivos móveis e centrais de atendimento emergencial. A solução foi desenvolvida com o objetivo de fornecer informações adicionais durante chamadas de emergência, permitindo que operadores tenham acesso a dados como localização, informações médicas e vídeos. A proposta da plataforma busca reduzir limitações em sistemas tradicionais de emergência, aumentando a disponibilidade de informações para as equipes responsáveis pelo atendimento e resposta às ocorrências (RAPIDSOS, 2025).

Apesar de ampliar a quantidade de dados disponíveis para os operadores, o *RapidSOS* não possui foco direto na interpretação automatizada da situação enfrentada pelo usuário através de inteligência artificial voltada ao redirecionamento contextual da ocorrência. Sua atuação está concentrada principalmente no compartilhamento de dados entre dispositivos e centrais emergenciais (RAPIDSOS, 2025).

A relação entre o *RapidSOS* e a proposta deste trabalho encontra-se principalmente na utilização de dados contextuais para auxiliar situações emergenciais. Ambos utilizam informações contextuais como localização geográfica, horário e descrição da ocorrência para auxiliar no direcionamento adequado da solicitação emergencial.

1.3.2 911 e 112

Representando modelos públicos integrados de emergência, os sistemas 911 e 112 constituem alguns dos principais exemplos de números universais de emergência utilizados internacionalmente. O número 911 é amplamente utilizado nos Estados

Unidos e Canadá como canal centralizado para acionamento de polícia, bombeiros e serviços médicos emergenciais, enquanto o 112 foi adotado como número padrão de emergência em diversos países da União Europeia (ASSOCIATION, 2024a; ASSOCIATION, 2024b).

Esses modelos possuem como principal característica a centralização do acesso aos serviços emergenciais através de um único número telefônico, reduzindo ambiguidades relacionadas à identificação do órgão responsável pela ocorrência. Além disso, sistemas modernos baseados em 911 e 112 passaram a incorporar recursos tecnológicos complementares, como compartilhamento automático de localização, integração digital entre centrais, suporte a comunicação multimodal e encaminhamento inteligente de chamadas (ASSOCIATION, 2024a; ASSOCIATION, 2024b).

Embora apresentem elevado nível de integração estrutural, esses sistemas dependem diretamente da infraestrutura pública de emergência existente em cada país, além de exigirem elevado nível de padronização tecnológica e administrativa para seu funcionamento eficiente. A relação entre esses sistemas e a proposta deste trabalho encontra-se principalmente na tentativa de centralizar e simplificar o acesso aos serviços emergenciais, reduzindo dificuldades relacionadas à identificação do órgão responsável pela ocorrência. Entretanto, diferentemente dos modelos tradicionais de 911 e 112, o projeto Centro de Ajuda propõe a utilização de inteligência artificial para interpretar contextualmente a situação relatada pelo usuário antes do direcionamento emergencial.

2 Revisão Bibliográfica

Para desenvolver um aplicativo capaz de utilizar inteligência artificial para interpretar situações emergenciais, compreender diferentes tipos de entrada de dados e auxiliar no direcionamento contextual de solicitações, torna-se necessário compreender os principais conceitos e tecnologias relacionados à área. Esta seção apresenta os fundamentos teóricos que sustentam o desenvolvimento da proposta, abordando conceitos de inteligência artificial, aprendizado de máquina, redes neurais, modelos de linguagem e processamento multimodal.

2.1 Inteligência Artificial

Segundo Russell e Norvig (2020), a inteligência artificial corresponde a uma área da computação voltada ao desenvolvimento de sistemas capazes de executar tarefas que normalmente exigiriam capacidades cognitivas humanas, como reconhecimento de padrões, interpretação de linguagem, tomada de decisão e resolução de problemas. Segundo sua abordagem, a inteligência artificial pode ser definida como o estudo de agentes inteligentes, isto é, sistemas capazes de perceber o ambiente ao seu redor e realizar ações com o objetivo de maximizar suas chances de sucesso em determinada tarefa.

2.1.1 Aprendizado de Máquina

A principal base para a inteligência artificial moderna é o aprendizado de máquina, também conhecido como *machine learning*. Essa área consiste no desenvolvimento de modelos computacionais capazes de aprender padrões e relações a partir de dados, permitindo que sistemas realizem previsões, classificações e tomadas de decisão sem depender exclusivamente de regras programadas manualmente.

Segundo Mitchell (1997), um programa é capaz de aprender a partir da experiência obtida durante a execução de uma tarefa, melhorando seu desempenho conforme

novos dados são analisados. Essa definição estabelece o aprendizado de máquina como um processo baseado em adaptação contínua, no qual os sistemas ajustam seus parâmetros internos de acordo com os padrões identificados durante o treinamento e utilização do modelo.

Esse conceito tornou-se um dos principais pilares da inteligência artificial devido à sua capacidade de identificar padrões complexos em grandes volumes de dados. Em vez de depender apenas de instruções fixas, modelos de aprendizado de máquina conseguem construir representações estatísticas capazes de reconhecer relações entre diferentes variáveis, permitindo aplicações em reconhecimento de voz, visão computacional, sistemas de recomendação, detecção de fraudes e processamento de linguagem natural (IBM, 2024).

De modo geral, os métodos de aprendizado de máquina podem ser divididos em três principais categorias: aprendizado supervisionado, aprendizado não supervisionado e aprendizado por reforço. No aprendizado supervisionado, o modelo é treinado utilizando exemplos rotulados, nos quais já existe uma resposta esperada associada aos dados de entrada. Essa abordagem é amplamente utilizada em tarefas de classificação e previsão. No aprendizado não supervisionado, os dados não possuem rótulos prévios, cabendo ao sistema identificar padrões, agrupamentos e relações ocultas nos dados analisados. Já o aprendizado por reforço consiste em um processo no qual agentes computacionais aprendem por tentativa e erro a partir de recompensas e penalidades recebidas durante interações com um ambiente (IBM, 2024).

2.1.2 Redes Neurais e Deep Learning

Entre as principais abordagens utilizadas no aprendizado de máquina moderno, destacam-se as redes neurais artificiais e as técnicas de *deep learning*. Essas estruturas tornaram-se responsáveis por grande parte dos avanços recentes da inteligência artificial,

especialmente em tarefas relacionadas ao reconhecimento de padrões complexos, interpretação de linguagem natural, visão computacional e processamento de áudio.

As redes neurais artificiais consistem em modelos computacionais inspirados no funcionamento biológico do cérebro humano. Sua estrutura é composta por neurônios artificiais interconectados, organizados em camadas responsáveis por receber, processar e transmitir informações. Durante o treinamento, os pesos dessas conexões são ajustados matematicamente com o objetivo de minimizar erros e melhorar a capacidade do modelo em reconhecer padrões presentes nos dados analisados (GOODFELLOW; BENGIO; COURVILLE, 2016).

Segundo Goodfellow, Bengio e Courville (2016), uma das principais características das redes neurais é sua capacidade de aprender representações hierárquicas dos dados. Isso significa que diferentes camadas da rede conseguem identificar níveis progressivamente mais complexos de informação. Em reconhecimento de imagens, por exemplo, camadas iniciais podem identificar linhas e bordas simples, enquanto camadas mais profundas passam a reconhecer formas, objetos e relações contextuais completas.

Dessa forma, o termo *deep learning*, também conhecido como aprendizado profundo, refere-se à utilização de redes neurais compostas por múltiplas camadas ocultas. O aumento da profundidade dessas arquiteturas permitiu avanços significativos no desempenho de modelos de inteligência artificial,bespecialmente em aplicações envolvendo grandes volumes de dados. Diferentemente de métodos tradicionais, nos quais características relevantes precisavam ser definidas manualmente, modelos de aprendizado profundo conseguem aprender automaticamente representações internas diretamente a partir dos dados utilizados durante o treinamento.

Eventualmente, diferentes arquiteturas de redes neurais passaram a ser desenvolvidas para resolver problemas específicos. Entre os principais exemplos estão as redes neurais convolucionais (Convolutional Neural Networks (CNNS)amplamente utilizadas em tarefas de visão computacional e reconhecimento de imagens, e as redes neurais

recorrentes (Recurrent Neural Networks (RNNS)) desenvolvidas para lidar com dados sequenciais, como textos e traduções automáticas (GOODFELLOW; BENGIO; COURVILLE, 2016).

Segundo Goodfellow, Bengio e Courville (2016), as redes neurais recorrentes possuem a capacidade de manter informações relacionadas a estados anteriores durante o processamento da sequência, permitindo que o modelo considere o contexto de elementos previamente analisados. Essa característica tornou as RNNS uma das principais abordagens utilizadas nas primeiras aplicações modernas de processamento de linguagem natural, reconhecimento de fala e tradução automática.

Apesar de representarem um avanço significativo em relação a modelos tradicionais, arquiteturas recorrentes apresentavam limitações relacionadas à dificuldade de processamento paralelo e à perda de informações em sequências muito longas, problema conhecido como *vanishing gradient*. Essas limitações impulsionaram o desenvolvimento de arquiteturas mais eficientes para interpretação contextual e processamento de grandes volumes de dados sequenciais (GOODFELLOW; BENGIO; COURVILLE, 2016).

2.1.3 Arquitetura Transformer

Diferente de arquiteturas anteriores, como RNNS, *Transformer* foi desenvolvido com o objetivo de processar informações de maneira paralela, reduzindo limitações relacionadas à dependência sequencial dos dados. Enquanto modelos recorrentes analisavam palavras e informações uma após a outra, a arquitetura *Transformer* passou a processar sequências inteiras simultaneamente, aumentando significativamente a eficiência computacional e a capacidade de treinamento em grandes conjuntos de dados (VASWANI *et al.*, 2017).

O principal componente responsável pelo funcionamento desta arquitetura é o mecanismo de atenção, especialmente o chamado *self-attention*. Esse mecanismo permite que o modelo avalie a relevância de diferentes elementos de uma sequência em relação aos

demais elementos presentes no mesmo contexto. Dessa forma, o *Transformer* consegue compreender relações semânticas entre palavras, identificar dependências contextuais distantes e interpretar significados de maneira mais consistente (VASWANI *et al.*, 2017).

Segundo Vaswani *et al.* (2017), Outra vantagem substancial da arquitetura *Transformer* é sua escalabilidade. O modelo demonstrou elevada capacidade de desempenho conforme aumentavam o volume de dados utilizados durante o treinamento e a quantidade de parâmetros presentes na rede neural. Essa característica contribuiu diretamente para o desenvolvimento de modelos modernos capazes de realizar tarefas complexas de interpretação contextual, geração de texto e raciocínio baseado em linguagem natural.

2.2 LLMs

A elevada escalabilidade da arquitetura *Transformer* possibilitou o desenvolvimento dos chamados modelos de linguagem de grande porte (Large Language Models (LLMS)). Esses modelos consistem em redes neurais treinadas em enormes volumes de dados textuais com o objetivo de identificar padrões estatísticos, relações semânticas e estruturas linguísticas presentes na linguagem humana..

Segundo Brown *et al.* (2020), os LLMS são capazes de aprender relações contextuais complexas entre palavras, frases e conceitos por meio do treinamento em grandes conjuntos de dados. Durante esse processo, os textos são divididos em *tokens*, que representam palavras completas ou fragmentos de palavras. Esses *tokens*, são convertidos em representações numéricas de alta dimensão, permitindo que o modelo analise matematicamente relações de proximidade e contexto entre diferentes elementos linguísticos.

Com base nessas relações estatísticas, os modelos conseguem calcular a probabilidade de um *token*, suceder outro dentro de uma sequência textual. Esse processo permite que LLMS realizem tarefas como geração de texto, tradução automática, sumarização,

interpretação contextual e respostas baseadas em linguagem natural. Quanto maior o volume de dados e a quantidade de parâmetros utilizados durante o treinamento, maior tende a ser a capacidade do modelo em compreender contextos complexos e produzir respostas coerentes (BROWN *et al.*, 2020).

2.2.1 Engenharia de Prompt

Para se comunicar com LLMS, é necessário fornecer instruções textuais conhecidas como *prompts*. Segundo Openai (2024), prompts consistem em instruções textuais utilizadas para orientar o comportamento do modelo durante a geração de respostas. Essas instruções podem incluir contexto adicional, regras de comportamento, exemplos de entrada e saída, restrições de formato e descrições detalhadas sobre a tarefa desejada. Quanto mais claro e contextualizado for o *prompt*, maior tende a ser a capacidade do modelo em produzir resultados coerentes com o objetivo esperado.

Essa interpretação contextual desempenha papel fundamental no funcionamento dos LLMS. Palavras isoladas podem possuir múltiplos significados dependendo do contexto em que aparecem. O termo banco, por exemplo, pode se referir tanto a uma instituição financeira quanto a um objeto utilizado para sentar. Nesse cenário, os modelos utilizam relações estatísticas aprendidas durante o treinamento para analisar palavras vizinhas, estruturas semânticas e padrões linguísticos capazes de indicar qual significado possui maior probabilidade dentro daquele contexto específico (OPENAI, 2024). Dessa forma, a maneira como as informações são apresentadas no *prompt*, influencia diretamente a interpretação realizada pelo modelo.

Para orientar essa interpretação, diferentes técnicas de engenharia de *prompt*, podem ser empregadas. Entre elas estão o *zero-shot*, no qual o modelo executa uma tarefa apenas com base na instrução fornecida; o *few-shot* que apresenta exemplos de entrada e saída para demonstrar o comportamento esperado; e o *chain-of-thought* que orienta a decomposição de problemas complexos em etapas intermediárias de raciocínio.

Por exemplo, para classificar uma emergência, o *prompt*, pode apenas solicitar a classificação (*zero-shot*), apresentar relatos previamente classificados (*few-shot*) ou solicitar que os elementos da ocorrência sejam analisados antes da resposta final (*chain-of--thought*) (WEI *et al.*, 2022).

Embora essas técnicas contribuam para orientar o modelo e tornar suas respostas mais previsíveis, elas não eliminam as limitações inerentes aos LLMS. Pequenas alterações na estrutura textual da instrução podem gerar respostas significativamente diferentes, especialmente em tarefas complexas ou ambíguas. Além disso, modelos de linguagem podem produzir informações incorretas, fenômeno frequentemente chamado de *hallucination* ("alucinação") no qual o sistema gera conteúdos sintaticamente coerentes, porém factualmente incorretos ou inexistentes (OPENAI, 2024).

Devido a essas características, a construção de prompts tornou-se uma etapa essencial no desenvolvimento de aplicações baseadas em inteligência artificial generativa. Estratégias como definição explícita de contexto, utilização de exemplos, delimitação de regras e restrição de formato são frequentemente utilizadas para aumentar a previsibilidade das respostas produzidas pelos modelos (OPENAI, 2024).

2.3 Modelos Multimodais

A evolução dos modelos de linguagem de grande porte também possibilitou o desenvolvimento dos chamados modelos multimodais, sistemas de inteligência artificial capazes de processar e interpretar múltiplos tipos de dados simultaneamente. Diferentemente de modelos tradicionais voltados exclusivamente para uma única modalidade de entrada, esses sistemas são treinados utilizando diferentes formas de informação, como linguagem natural, imagens, áudio e vídeo (OPENAI, 2023).

Tais modelos possuem a capacidade de integrar diferentes modalidades em uma mesma representação contextual, permitindo compreender relações entre informações textuais, visuais e sonoras presentes nos dados analisados. Como resultado, essa abor-

dagem amplia significativamente a capacidade interpretativa da inteligência artificial, tornando possível a execução de tarefas mais complexas que envolvem múltiplos formatos de informação (OPENAI, 2023).

Segundo Openai (2023), o funcionamento desses modelos geralmente ocorre através da conversão das diferentes modalidades em representações numéricas compatíveis dentro do espaço vetorial utilizado pela rede neural. Dessa forma, textos, imagens, sons e vídeos podem ser processados conjuntamente pelo modelo, permitindo que relações contextuais entre diferentes tipos de informação sejam identificadas durante a inferência.

A utilização de arquiteturas multimodais tornou-se especialmente relevante em aplicações modernas relacionadas à análise de imagens, reconhecimento de fala, interpretação audiovisual e assistentes inteligentes capazes de compreender diferentes formatos de entrada simultaneamente. Além disso, esses modelos passaram a demonstrar capacidade de realizar tarefas cruzadas entre modalidades, como descrever imagens em linguagem natural, responder perguntas sobre conteúdos visuais e interpretar informações sonoras contextualizadas (OPENAI, 2023).

2.3.1 Processamento de Imagem

Diferente de LLMS tradicionais voltadas exclusivamente para linguagem natural, os modelos multimodais possuem a capacidade de interpretar conteúdos visuais e relacioná-los ao contexto textual fornecido pelo usuário. Essa capacidade ampliou significativamente o potencial de aplicações baseadas em inteligência artificial, especialmente em cenários nos quais informações visuais desempenham papel importante para compreensão da situação analisada. Modelos multimodais modernos conseguem identificar objetos, interpretar cenas, reconhecer padrões visuais e responder perguntas relacionadas ao conteúdo presente em imagens, permitindo que informações textuais e visuais sejam analisadas conjuntamente durante a inferência (OPENAI, 2023).

Segundo Openai (2023), o funcionamento do processamento visual nesses modelos ocorre através da conversão dos elementos presentes na imagem em representações numéricas compatíveis com o espaço vetorial utilizado pela rede neural. Durante esse processo, características como formas, cores, texturas, objetos e relações espaciais são transformadas em *embeddings* (representações vetoriais que preservam informações semânticas e estruturais dos dados) capazes de serem interpretados juntamente com dados textuais e contextuais. Dessa forma, o modelo consegue estabelecer relações semânticas entre diferentes modalidades de entrada, permitindo interpretação contextual integrada entre imagem e linguagem natural.

O processamento de imagens em modelos multimodais possui forte relação com técnicas de visão computacional RNNS anteriormente citados no documento. Segundo Goodfellow, Bengio e Courville (2016), arquiteturas convolucionais foram desenvolvidas especificamente para identificar padrões visuais em imagens através da aplicação sucessiva de filtros matemáticos capazes de reconhecer características progressivamente mais complexas. Em camadas iniciais, o modelo pode identificar bordas, linhas e texturas simples, enquanto camadas mais profundas conseguem reconhecer objetos completos, estruturas e relações contextuais presentes na cena analisada.

Embora modelos multimodais modernos não dependam exclusivamente de CNNS tradicionais, muitas arquiteturas atuais ainda utilizam princípios derivados do aprendizado profundo aplicado à visão computacional para interpretar informações visuais. Em sistemas baseados em *Transformers* multimodais, por exemplo, imagens podem ser divididas em pequenas regiões chamadas *patches*, que são convertidas em tokens visuais semelhantes aos tokens textuais utilizados em modelos de linguagem natural. Dessa forma, o modelo consegue aplicar mecanismos de atenção contextual também sobre elementos visuais da imagem (DOSOVITSKIY *et al.*, 2021).

2.3.2 Processamento de Áudio

O processamento de áudio em modelos multimodais permite que sistemas de inteligência artificial interpretem informações sonoras e relacionem esses dados ao contexto textual e visual fornecido durante a inferência. Essa capacidade tornou-se especialmente relevante em aplicações modernas envolvendo reconhecimento de fala, assistentes virtuais, transcrição automática e interpretação contextual de sons presentes no ambiente analisado.

Diferente do processamento de imagens, no qual os elementos visuais podem ser examinados de forma espacial, o áudio é composto por informações distribuídas sequencialmente ao longo do tempo. Por esse motivo, modelos destinados à interpretação sonora precisam considerar tanto características individuais do sinal acústico quanto as dependências temporais existentes entre diferentes trechos da gravação. Segundo Radford *et al.* (2022), sistemas modernos de processamento de áudio convertem sinais sonoros em representações numéricas adequadas ao processamento por redes neurais. Nesse procedimento, atributos como frequência, intensidade, duração e variações temporais são codificados em *embeddings*, possibilitando a associação dessas informações com outras modalidades presentes no sistema multimodal.

Grande parte das aplicações contemporâneas relacionadas à análise de áudio utiliza técnicas de reconhecimento automático de fala (Automatic Speech Recognition (ASR), responsáveis por transformar a linguagem falada em texto para posterior interpretação pelos modelos de linguagem. Esses sistemas normalmente executam etapas como aquisição do sinal acústico, extração de características relevantes e decodificação da fala reconhecida Radford *et al.* (2022). Embora frequentemente integrem soluções baseadas em LLMS, mecanismos de ASR constituem componentes independentes e especializados, atuando como intermediários entre o conteúdo sonoro e os modelos responsáveis pela compreensão textual.

As primeiras aplicações modernas de reconhecimento de fala utilizaram fortemente arquiteturas recorrentes (RNNS, anteriormente citadas neste documento. Segundo Goodfellow, Bengio e Courville (2016), essas redes são capazes de preservar informações provenientes de etapas anteriores do processamento, característica especialmente útil para a modelagem de sequências temporais presentes na fala humana.

2.3.3 Processamento de Vídeo

O processamento de vídeo em modelos multimodais envolve a análise simultânea de informações visuais e temporais, permitindo que sistemas de inteligência artificial compreendam eventos que se desenvolvem ao longo do tempo. Ao contrário de imagens estáticas, vídeos apresentam uma sequência contínua de quadros, exigindo que o modelo identifique não apenas os elementos presentes em cada cena, mas também as transformações e interações que ocorrem durante sua evolução temporal.

De modo geral, a análise de vídeos inicia-se pela decomposição do conteúdo audiovisual em múltiplos quadros de imagem, que podem ser processados individualmente por mecanismos semelhantes aos utilizados em tarefas de visão computacional. Segundo Openai (2023), as características extraídas desses quadros são convertidas em representações adequadas ao processamento neural, enquanto informações relacionadas à dinâmica temporal da sequência são utilizadas para identificar movimentos, ações e mudanças de contexto.

Além da dimensão visual, muitos sistemas multimodais também analisam o áudio associado ao vídeo. Essa integração permite correlacionar falas, ruídos e demais eventos acústicos com os elementos observados em cena, contribuindo para uma compreensão mais completa do conteúdo. Em diversos cenários, a combinação dessas modalidades fornece informações que não poderiam ser obtidas por meio da análise isolada de apenas uma delas Openai (2023).

Arquiteturas modernas baseadas em *Transformers* multimodais demonstraram elevado desempenho nesse domínio devido à capacidade de aplicar mecanismos de atenção sobre diferentes regiões espaciais e instantes temporais da sequência audiovisual. Como consequência, o modelo consegue relacionar objetos, ações, diálogos e acontecimentos distribuídos ao longo do vídeo, favorecendo uma compreensão contextual mais abrangente Vaswani *et al.* (2017).

3 Metodologia

A metodologia utilizada neste trabalho consiste na análise de requisitos para o desenvolvimento de uma prova de conceito funcional de um sistema inteligente voltado ao auxílio no redirecionamento de solicitações emergenciais. A proposta envolve a construção de uma aplicação móvel integrada a um servidor responsável pelo recebimento, processamento e encaminhamento contextual das informações enviadas pelo usuário. O sistema foi estruturado utilizando arquitetura cliente-servidor, permitindo separar as responsabilidades relacionadas à interface da aplicação, processamento das requisições e integração com modelos de inteligência artificial.

3.1 Critérios de Escolha para o(s) Modelo(s) LLM

O funcionamento da proposta apresentada neste trabalho depende diretamente da integração entre o servidor *backend* e modelos de inteligência artificial responsáveis pela interpretação contextual das solicitações enviadas pelos usuários. Dessa forma, tornou-se necessário analisar diferentes modelos de linguagem de grande porte capazes de processar informações multimodais e fornecer respostas contextualizadas adequadas ao cenário emergencial proposto pelo sistema Centro de Ajuda.

Considerando as necessidades da aplicação, foram definidos critérios técnicos para a seleção dos modelos avaliados. O primeiro critério foi a capacidade de processamento de diferentes modalidades de entrada, incluindo texto, áudio e imagem. Essa característica é necessária porque o sistema foi projetado para permitir que o usuário relate uma ocorrência utilizando diferentes formas de comunicação, conforme a situação enfrentada. Em um cenário de emergência, por exemplo, um usuário pode enviar apenas uma fotografia do local, um áudio descrevendo os acontecimentos ou uma mensagem textual, tornando indispensável que o modelo seja capaz de interpretar qualquer uma dessas modalidades.

Outro critério analisado foi a disponibilidade de utilização em infraestrutura de nuvem. Essa característica foi considerada devido às limitações técnicas associadas à execução local de modelos de linguagem de grande porte. Modelos multimodais modernos demandam elevada capacidade computacional, grande quantidade de memória e, em muitos casos, aceleradores gráficos especializados, recursos que normalmente não estão disponíveis em computadores pessoais convencionais. Dessa forma, a disponibilização dos modelos por meio de Application Programming Interfaces (APIS) em nuvem permite que todo o processamento seja realizado em infraestrutura especializada, tornando sua utilização viável na aplicação proposta sem exigir *hardware* de alto desempenho no servidor ou nos dispositivos dos usuários.

Também foi avaliada a capacidade de realização de pesquisas na internet. Embora a interpretação inicial das ocorrências seja baseada nas informações fornecidas pelo usuário, determinadas situações podem exigir a consulta de informações atualizadas durante o processamento da solicitação. Serviços de atendimento, unidades de saúde e demais recursos de apoio podem sofrer alterações temporárias de funcionamento, indisponibilidade, mudanças de endereço, contatos ou outras condições que não estão necessariamente presentes na base de conhecimento do modelo. Dessa forma, a capacidade de realizar pesquisas na internet permite complementar a resposta com informações mais recentes e potencialmente mais adequadas ao contexto da ocorrência.

Entre os modelos analisados destacam-se o GPT-5.5, desenvolvido pela OpenAI; o Claude Opus 4.8, desenvolvido pela Anthropic; o Gemini 3.5 Flash, desenvolvido pela Google; o Llama 4 Maverick, disponibilizado pela Meta; o Qwen 3.6 Plus, desenvolvido pela Alibaba Cloud; o Nemotron 3 Nano Omni 30B, desenvolvido pela NVIDIA; e o North Mini Code, desenvolvido pela Cohere. As informações apresentadas e as comparações entre os modelos correspondem à análise realizada em 10 de junho de 2026.

A seleção inicial desses modelos ocorreu devido à relevância no cenário atual da inteligência artificial generativa, ao suporte parcial ou completo às funcionalidades

exigidas pelo projeto e à disponibilidade de integração por meio de APIS comerciais compatíveis com aplicações executadas em nuvem. Entretanto, nem todos os modelos avaliados atenderam integralmente aos critérios técnicos estabelecidos para a proposta desenvolvida neste trabalho. Por fim, esses critérios serviram como base para a análise comparativa apresentada nas subseções seguintes, nas quais cada modelo foi avaliado quanto ao atendimento dos requisitos técnicos definidos para o sistema.

A partir dos critérios estabelecidos, foi elaborada uma comparação preliminar entre os modelos selecionados. A tabela a seguir apresenta uma síntese do atendimento de cada modelo aos requisitos de multimodalidade, disponibilidade em nuvem e capacidade de pesquisa na internet, permitindo visualizar as principais diferenças identificadas antes do detalhamento individual.

Tabela 1 – Comparação de modelos quanto aos critérios técnicos

Critério	**GPT**	**Claude**	**Llama**	**Qwen**	**Gemini**	**Nemotron**	**North**
Multimodal	Parcial	Parcial	Parcial	Parcial	Sim	Parcial	Não
Nuvem	Sim	Sim	Sim	Sim	Sim	Sim	Sim
Pesquisa	Sim	Sim	Sim	Sim	Sim	Não	Não

Elaborado pelo próprio autor (2026).

Os resultados sintetizados na tabela oferecem uma visão geral das capacidades identificadas, mas não demonstram isoladamente as particularidades e limitações de cada modelo. Por esse motivo, as subseções seguintes apresentam as análises individuais, descrevendo como cada solução atende total ou parcialmente aos critérios estabelecidos e justificando as classificações atribuídas na comparação.

3.1.1 GPT-5.5

O GPT-5.5, desenvolvido pela OpenAI, foi selecionado para análise devido à sua relevância no cenário atual da inteligência artificial generativa. A avaliação do modelo foi realizada com base nos critérios técnicos definidos para a seleção das LLMS utilizadas na proposta deste trabalho. (OPENAI, 2026)

Em relação ao critério de capacidade de processamento de diferentes modalidades de entrada, o GPT-5.5 oferece suporte ao processamento de texto, imagens e áudio por meio dos serviços disponibilizados pela plataforma da OpenAI, permitindo que aplicações interpretem diferentes modalidades de entrada de acordo com a forma como as informações são fornecidas pelo usuário. (OPENAI, 2026)

Quanto ao suporte à interpretação multimodal, o GPT-5.5 apresenta recursos para processamento de diferentes modalidades de entrada por meio da plataforma da OpenAI. Entretanto, sua Application Programming Interface (API) não disponibiliza, de forma nativa, o envio simultâneo de texto, imagem e áudio em uma única solicitação, exigindo a utilização de recursos complementares da plataforma para integrar essas modalidades. Dessa forma, o modelo atende parcialmente ao critério de multimodalidade definido para este trabalho. (OPENAI, 2026)

Em relação à infraestrutura, o GPT-5.5 é disponibilizado por meio de APIS em nuvem oferecidas pela OpenAI, podendo também ser acessado por plataformas intermediárias. Essa abordagem dispensa a execução local do modelo e permite sua utilização em aplicações que não dispõem de hardware especializado para inferência de modelos de grande porte. (OPENAI, 2026)

Por fim, o GPT-5.5 oferece suporte à realização de pesquisas na internet por meio de funcionalidades de acesso à web disponibilizadas pela OpenAI, permitindo complementar seu conhecimento com informações atualizadas quando necessário. Essa capacidade atende ao critério estabelecido para o sistema proposto. (OPENAI, 2026)

3.1.2 Claude Opus 4.8

Desenvolvido pela Anthropic, o Claude Opus 4.8 foi incluído na análise por representar uma das principais famílias de modelos de linguagem disponíveis comercialmente. Sua avaliação considerou os critérios técnicos estabelecidos para verificar sua adequação à proposta do sistema Centro de Ajuda. (ANTHROPIC, 2026)

Segundo Anthropic (2026), o Claude Opus 4.8 aceita como entrada informações textuais e imagens, além de documentos compatíveis com essas modalidades. Entretanto, o modelo não disponibiliza processamento nativo de áudio como entrada em sua API, fazendo com que atenda apenas parcialmente ao critério de suporte às modalidades de entrada adotado neste trabalho.

No que se refere à interpretação multimodal, o modelo consegue combinar informações provenientes de texto e imagem durante uma mesma interação, produzindo respostas contextualizadas a partir dessas entradas. Contudo, a ausência de suporte nativo ao processamento simultâneo de áudio limita sua capacidade de atender integralmente ao requisito de multimodalidade definido para a proposta. (ANTHROPIC, 2026)

A documentação da Anthropic informa que o Claude Opus 4.8 também é disponibilizado por meio de APIS executadas em infraestrutura de nuvem, podendo ser facilmente integrado à aplicação, tornando sua utilização compatível com a arquitetura adotada neste trabalho. (ANTHROPIC, 2026)

Por fim, a plataforma Claude disponibiliza recursos de pesquisa na internet, permitindo que o modelo consulte informações externas durante a geração das respostas quando essa funcionalidade é utilizada pela aplicação. Dessa forma, o modelo atende ao critério relacionado ao acesso a informações atualizadas estabelecido nesta pesquisa. (ANTHROPIC, 2026)

3.1.3 Llama 4 Maverick

O Llama 4 Maverick, desenvolvido pela Meta, foi incluído na análise por integrar uma das principais famílias de modelos de linguagem com pesos abertos disponíveis atualmente. Sua avaliação foi conduzida considerando os critérios técnicos definidos para verificar sua adequação aos requisitos do sistema Centro de Ajuda. (META, 2026a)

De acordo com Meta (2026a), o Llama 4 Maverick oferece suporte ao processamento de texto e imagens como modalidades de entrada. Entretanto, a documentação

do modelo não prevê processamento nativo de áudio, fazendo com que o atendimento ao critério de suporte às modalidades de entrada seja parcial em relação às necessidades estabelecidas neste trabalho.

Em relação à interpretação multimodal, o modelo permite analisar conjuntamente informações textuais e visuais, utilizando essas modalidades para produzir respostas contextualizadas. Apesar dessa capacidade, a ausência de suporte nativo ao processamento simultâneo de áudio impede que o modelo atenda integralmente ao requisito de multimodalidade adotado como critério de seleção. (META, 2026a)

A Meta disponibiliza o Llama 4 Maverick para utilização em infraestrutura de nuvem por meio de provedores e APIS compatíveis, além de permitir sua integração com plataformas de distribuição de Inteligências Artificiais (IAS). Por sua utilização não depender da execução local do modelo, sua adoção é compatível com a arquitetura cliente-servidor proposta neste trabalho. (META, 2026b)

Quanto ao critério relacionado à pesquisa na internet, o Llama 4 Maverick pode ser integrado a ferramentas externas de busca e recuperação de informações, possibilitando que aplicações obtenham dados atualizados durante o processamento das solicitações. Essa funcionalidade depende da integração realizada pela aplicação, não sendo uma capacidade nativa do modelo. (META, 2026a)

3.1.4 Qwen 3.6 Plus

O Qwen 3.6 Plus, desenvolvido pela Alibaba Cloud, foi incluído nesta análise por representar uma das principais soluções proprietárias da família Qwen. Sua avaliação teve como objetivo verificar o atendimento aos critérios técnicos definidos para a seleção das LLMS empregadas no sistema Centro de Ajuda. (CLOUD, 2026a)

Segundo Cloud (2026a), o Qwen 3.6 Plus disponibiliza processamento nativo de texto e imagens, além de recursos voltados à compreensão de documentos e conteúdo visual. Entretanto, o modelo não oferece processamento nativo de áudio como modali-

dade de entrada, atendendo apenas parcialmente ao critério relacionado às modalidades de entrada estabelecido neste trabalho.

No que diz respeito à interpretação multimodal, o modelo foi projetado para correlacionar informações provenientes de texto e imagens durante uma mesma interação, permitindo análises contextuais integradas dessas modalidades. Apesar desse suporte, a ausência de processamento nativo de áudio impede que o requisito de multimodalidade definido para a proposta seja atendido integralmente. (CLOUD, 2026b)

A utilização do Qwen 3.6 Plus ocorre por meio da infraestrutura em nuvem disponibilizada pela Alibaba Cloud, sendo acessível por APIS compatíveis com diferentes padrões de integração. Essa característica elimina a necessidade de execução local do modelo e favorece sua adoção em arquiteturas cliente-servidor semelhantes à utilizada neste trabalho. (CLOUD, 2026a)

Quanto ao critério de pesquisa na internet, o ecossistema Qwen permite a integração do modelo com ferramentas externas e recursos de busca durante a execução de tarefas, possibilitando a obtenção de informações atualizadas quando configurado para esse tipo de operação. Dessa forma, o modelo atende ao critério definido para esta pesquisa. (CLOUD, 2026a)

3.1.5 Gemini 3.5 Flash

O Gemini 3.5 Flash, desenvolvido pela Google, foi selecionado para análise em razão de sua relevância entre os modelos multimodais atuais e da disponibilidade de acesso por meio de APIS compatíveis com a proposta deste trabalho. Sua avaliação foi conduzida considerando os critérios técnicos estabelecidos para a seleção das LLMS empregadas no sistema Centro de Ajuda. (GOOGLE, 2026a) Segundo Google (2026a), o Gemini 3.5 Flash oferece suporte nativo ao processamento de texto, imagens, áudio, vídeo e documentos, permitindo que diferentes modalidades de entrada sejam utilizadas

durante uma mesma interação. Dessa forma, o modelo atende integralmente ao critério relacionado ao processamento das modalidades de entrada adotado nesta pesquisa.

No que se refere à interpretação multimodal, o Gemini 3.5 Flash foi projetado para correlacionar simultaneamente informações provenientes das modalidades suportadas, possibilitando uma compreensão contextual unificada das entradas recebidas. Essa característica atende integralmente ao requisito de multimodalidade definido para o sistema proposto. (GOOGLE, 2026a)

Além dos modelos anteriores, a documentação da Google também informa que o Gemini 3.5 Flash é disponibilizado por meio de APIS executadas em infraestrutura de nuvem e está disponível em plataformas intermediárias. (GOOGLE, 2026b)

Além disso, o Gemini 3.5 Flash disponibiliza recursos nativos de pesquisa na internet por meio das ferramentas da plataforma Gemini, permitindo consultar informações atualizadas durante o processamento das solicitações. Essa funcionalidade atende ao critério de acesso a informações externas estabelecido para a avaliação dos modelos. (GOOGLE, 2026a)

3.1.6 Nemotron 3 Nano Omni

O Nemotron 3 Nano Omni 30B A3B Reasoning, desenvolvido pela NVIDIA, foi incluído na análise por representar um modelo multimodal voltado a aplicações que exigem raciocínio sobre diferentes tipos de dados. Sua avaliação foi realizada considerando os critérios técnicos definidos para a seleção das LLMS empregadas na proposta do sistema Centro de Ajuda. (NVIDIA, 2026)

Conforme a documentação oficial da NVIDIA, o modelo aceita como entrada texto, imagens, áudio e vídeo, permitindo que diferentes modalidades sejam utilizadas durante uma mesma interação. Dessa forma, o Nemotron 3 Nano Omni atende integralmente ao critério relacionado ao processamento de diferentes modalidades de entrada adotado nesta pesquisa. (NVIDIA, 2026)

Em relação ao suporte à interpretação multimodal, o modelo foi projetado para realizar raciocínio conjunto sobre informações provenientes das modalidades suportadas em um único processo de inferência. Essa característica permite correlacionar simultaneamente diferentes fontes de informação, atendendo ao requisito de multimodalidade estabelecido para o sistema proposto. (NVIDIA, 2026)

A NVIDIA disponibiliza o Nemotron 3 Nano Omni por meio de serviços executados em infraestrutura de nuvem, incluindo APIS NVIDIA NIM e provedores parceiros. Essa abordagem possibilita sua integração à arquitetura cliente-servidor adotada neste trabalho sem exigir a execução local do modelo. (NVIDIA, 2026)

Quanto ao critério de pesquisa na internet, o Nemotron 3 Nano Omni não disponibiliza um mecanismo nativo de consulta à *web*. Entretanto, sua arquitetura permite integração com ferramentas externas responsáveis pela recuperação de informações atualizadas durante a execução da aplicação, possibilitando o atendimento deste requisito quando tais recursos são incorporados ao sistema. (NVIDIA, 2026)

3.1.7 North Mini Code

O North Mini Code, desenvolvido pela Cohere, participou desta análise por ocupar espaço como um modelo de peso, voltado à geração de código e à execução de tarefas de engenharia de software por agentes. O modelo utiliza uma arquitetura *Mixture of Experts*, com 30 bilhões de parâmetros totais e 3 bilhões ativos durante a inferência. Sua avaliação foi realizada considerando os critérios técnicos definidos para verificar sua adequação à proposta do sistema Centro de Ajuda. (COHERE, 2026)

Sobre às modalidades de entrada, o North Mini Code foi desenvolvido essencialmente para o processamento de texto e código. A documentação não indica suporte nativo à utilização de imagens ou áudio como entrada, infelizmente fazendo com que este modelo atende apenas ao processamento textual entre as modalidades adotadas neste trabalho. (COHERE, 2026)

Quanto a sua interpretação multimodal, a ausência de suporte nativo a imagens e áudio impede que o modelo correlacione essas informações com o conteúdo textual em uma mesma solicitação. Dessa forma, embora consiga interpretar relatos fornecidos em texto, o North Mini Code não atende ao requisito de multimodalidade estabelecido para o sistema proposto. (COHERE, 2026)

O modelo é disponibilizado sob a licença Apache 2.0 e pode ser executado localmente, devido à quantidade reduzida de parâmetros ativos, ou acessado por serviços em nuvem oferecidos pela Cohere e por plataformas intermediárias. (COHERE, 2026)

Quanto ao critério de pesquisa na internet, o North Mini Code não possui um mecanismo nativo de consulta à *web*. Entretanto, seu suporte ao uso de ferramentas permite que seja integrado a mecanismos externos de busca e recuperação de informações. Assim, o atendimento a esse critério depende dos recursos fornecidos pela aplicação, não constituindo uma capacidade nativa do modelo. (COHERE, 2026)

3.2 Requisitos do Frontend

O *frontend* corresponde à camada da aplicação responsável pela interação direta com o usuário. Em sistemas baseados em arquitetura cliente-servidor, essa camada tem como função apresentar informações, coletar dados de entrada e encaminhar solicitações aos serviços disponibilizados pelo servidor. No contexto deste trabalho, o *frontend* é responsável por permitir que o usuário registre uma ocorrência emergencial por meio de texto, áudio ou imagem, além de disponibilizar os resultados retornados pelo sistema de classificação inteligente.

3.2.1 Infraestrutura Frontend

A infraestrutura *frontend* da aplicação foi definida considerando os elementos necessários para o funcionamento da prova de conceito. Para que o usuário possa registrar uma ocorrência, enviar as informações ao sistema e visualizar o resultado da

classificação, foram identificadas três áreas fundamentais: interface do usuário, compatibilidade multiplataforma e comunicação cliente-servidor.

A interface do usuário é responsável pela apresentação visual da aplicação e pela coleta das informações fornecidas pelo usuário. A compatibilidade multiplataforma possibilita o acesso à aplicação em diferentes dispositivos e sistemas operacionais. Já a comunicação cliente-servidor realiza a troca de informações entre o *frontend* e os serviços executados no *backend*, permitindo o envio dos dados coletados e o recebimento dos resultados gerados pelo sistema.

3.2.2 Interface do Usuário

A construção da interface da aplicação foi realizada utilizando HTML, CSS e JavaScript, tecnologias consolidadas como padrão para o desenvolvimento de aplicações *web*. Sua adoção ocorreu devido à ampla compatibilidade entre navegadores, à extensa documentação disponível, ao ecossistema consolidado de ferramentas e à possibilidade de reutilização do código em soluções híbridas para dispositivos móveis, reduzindo o esforço de desenvolvimento e manutenção da aplicação.

Nesse contexto, o HTML foi empregado para estruturar semanticamente a interface, o CSS para definir sua apresentação visual e adaptação a diferentes tamanhos de tela, enquanto o JavaScript foi utilizado para implementar a lógica da aplicação, gerenciar eventos, manipular dinamicamente os elementos da interface e realizar a comunicação com os serviços disponibilizados pelo *backend*.

3.2.3 Compatibilidade Multiplataforma

A compatibilidade multiplataforma da aplicação é proporcionada pela utilização de tecnologias *web*, permitindo que a interface seja acessada em diferentes dispositivos e sistemas operacionais por meio de navegadores compatíveis. Essa abordagem evita a necessidade de desenvolver uma versão específica da aplicação para cada plataforma.

Além disso, a aplicação pode ser disponibilizada como uma Progressive Web App (PWA). Uma PWA consiste em uma aplicação web que incorpora características de aplicações instaláveis, podendo ser adicionada à tela inicial do dispositivo e executada em uma janela própria. Sua configuração básica utiliza recursos como um arquivo de manifesto, responsável por definir informações como nome, ícone e aparência da aplicação, e um *Service Worker*, que pode gerenciar o armazenamento de recursos e determinadas funcionalidades em segundo plano.

Essa abordagem mostrou-se adequada à prova de conceito por permitir sua utilização em computadores e dispositivos móveis sem depender de tecnologias de empacotamento nativo, preservando uma única base de código desenvolvida com HTML, CSS e JavaScript.

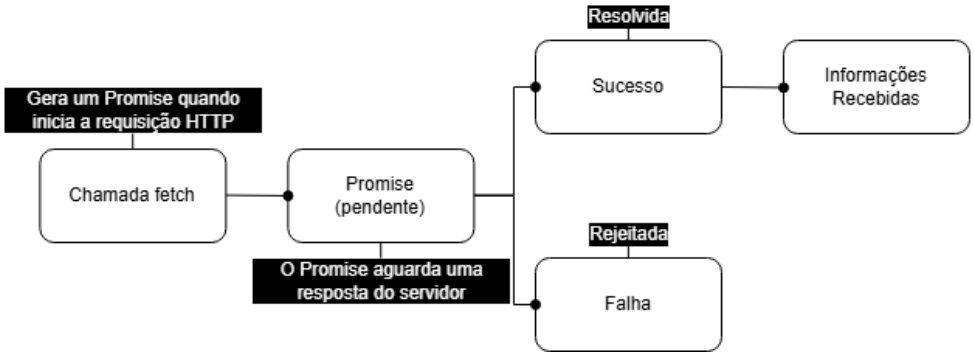
3.2.4 Comunicação Cliente-Servidor

A comunicação entre *frontend* e *backend* foi implementada utilizando a API Fetch, interface nativa disponibilizada pelos navegadores modernos para a realização de requisições HTTP assíncronas. Conforme apresentado em Figura 2, ao executar a função

`fetch()`

, a aplicação inicia a requisição HTTP e recebe imediatamente uma Promise, inicialmente em estado pendente. Enquanto a requisição é processada, a execução da interface permanece disponível, sem precisar aguardar de forma bloqueante pela resposta do servidor. Quando a comunicação é concluída com sucesso, a Promise é resolvida, permitindo o acesso à resposta e às informações retornadas pelo *backend*. Caso ocorra uma falha que impeça a conclusão da requisição, a Promise é rejeitada, possibilitando que a aplicação identifique e trate o erro. Dessa forma, o uso da APIS Fetch contribui para uma experiência de utilização mais fluida e responsiva.

Figura 2 – Fluxo da API Fetch com Promise



Elaborado pelo próprio autor (2026).

Nesse contexto, a API Fetch é responsável pelo envio das informações fornecidas pelo usuário, incluindo dados textuais, arquivos multimídia e informações contextuais, bem como pelo recebimento das respostas geradas pelos serviços de processamento executados no *backend*.

3.3 Requisitos do Backend

O *backend* concentra o processamento necessário para transformar os dados enviados pelo usuário em uma resposta utilizável pela aplicação. Para viabilizar a prova de conceito do sistema Centro de Ajuda, essa camada precisava cumprir quatro funções principais: receber solicitações contendo diferentes modalidades de entrada, complementar os dados com informações de localização, encaminhar o contexto da ocorrência ao modelo de linguagem e devolver ao *frontend* uma resposta estruturada.

A centralização dessas operações no servidor evita que a aplicação cliente precise conhecer os serviços externos utilizados ou armazenar credenciais de acesso às APIS. Dessa forma, o *frontend* permanece responsável apenas pela coleta e apresentação das informações, enquanto validações, integrações e regras relacionadas à classificação das ocorrências são executadas no *backend*.

3.3.1 Infraestrutura Backend

A infraestrutura foi construída como uma aplicação *web* executada em servidor e organizada em torno de uma rota principal de atendimento. Essa rota recebe o relato do usuário, valida os dados enviados e coordena as etapas necessárias até a obtenção da classificação. Durante esse processo, o servidor comunica-se com o serviço de geolocalização, prepara o contexto utilizado na consulta ao modelo de linguagem e trata a resposta antes de devolvê-la à aplicação cliente.

Para implementar esse fluxo, foram utilizados o Node.js como ambiente de execução, o Express para definição e controle das rotas HTTP e a API Fetch para comunicação com os serviços externos. Cada uma dessas tecnologias exerce uma função específica na infraestrutura, permitindo separar o recebimento da solicitação, o processamento interno e as integrações realizadas pela aplicação.

3.3.2 Ambiente de Execução

O *backend* foi implementado em Node.js, possibilitando a utilização de JavaScript também no lado do servidor. Sua adoção permitiu manter uma base tecnológica próxima à utilizada no *frontend* e simplificar a manipulação das estruturas JSON trocadas entre as camadas da aplicação.

O modelo assíncrono do Node.js foi especialmente relevante para a prova de conceito, pois parte significativa do tempo de processamento depende de serviços externos. Durante uma solicitação, o servidor pode aguardar a conversão das coordenadas geográficas e a resposta do modelo de linguagem sem bloquear completamente a execução de outras operações. Esse comportamento é adequado a uma aplicação cuja principal função consiste em coordenar requisições HTTP e organizar os resultados obtidos.

Na implementação desenvolvida, o Node.js executa o servidor, recebe os dados encaminhados pelo *frontend*, controla a sequência de chamadas externas e prepara a resposta final da ocorrência.

3.3.3 Estrutura da Aplicação

Sobre o ambiente Node.js, o Express foi utilizado para criar a interface HTTP do *backend*. Sua função principal na aplicação é disponibilizar a rota responsável pelo recebimento das solicitações de ajuda e fornecer os mecanismos necessários para acessar o conteúdo enviado pelo *frontend*.

Quando uma requisição é recebida, o Express direciona os dados para a função responsável pelo processamento da ocorrência. Antes de prosseguir, são verificadas informações obrigatórias, como a presença e a validade das coordenadas geográficas. Em seguida, a solicitação percorre as etapas de obtenção da localização, construção dos prompts, consulta ao modelo de linguagem e tratamento da resposta retornada.

A utilização do Express também permitiu manter o controle do fluxo HTTP em um ponto definido da aplicação. Assim, situações como dados inválidos, falhas nas APIS externas ou respostas incompatíveis podem ser convertidas em respostas HTTP adequadas, evitando que erros internos sejam enviados diretamente ao *frontend*.

3.3.4 Integração com Serviços Externos

As chamadas realizadas pelo *backend* foram implementadas com a API Fetch, disponível nativamente nas versões modernas do Node.js. Diferentemente de sua utilização no *frontend*, onde é responsável por enviar a solicitação de ajuda, no servidor a Fetch estabelece a comunicação com as APIS necessárias para complementar e analisar os dados recebidos.

Uma dessas integrações ocorre com o serviço de geocodificação reversa do Google Maps. Por meio dele, as coordenadas de latitude e longitude são convertidas em informações textuais, como rua, bairro, cidade e estado. Essa conversão fornece ao modelo

de linguagem um contexto de localização mais compreensível do que a utilização isolada dos valores numéricos.

A Fetch também é utilizada para encaminhar à API os prompts e as modalidades presentes na solicitação. A plataforma direciona o conteúdo ao modelo selecionado e retorna o resultado da análise ao *backend*. Como ambas as operações dependem de serviços remotos, o uso de requisições assíncronas permite controlar cada etapa, verificar falhas de comunicação e impedir que uma resposta incompleta seja tratada como resultado válido.

3.4 Fluxo de Comunicação

O fluxo de comunicação inicia-se quando o *frontend* envia uma solicitação contendo uma ou mais formas de relato, que podem incluir texto, áudio e imagem. Junto a esse conteúdo, são enviadas as coordenadas geográficas e as demais informações contextuais coletadas pela aplicação.

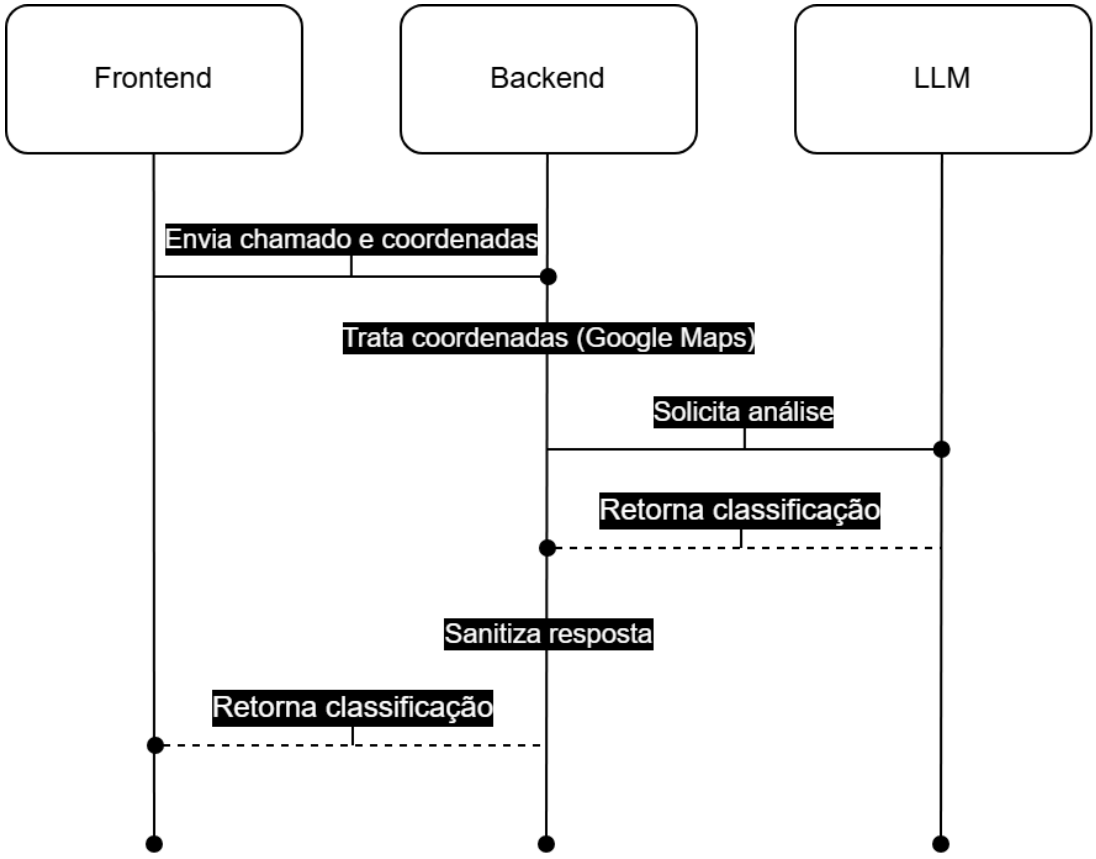
Ao receber a requisição, o *backend* verifica os dados necessários para o processamento. As coordenadas são encaminhadas à API Google Maps para realização da geocodificação reversa, resultando em uma descrição textual da localização. Posteriormente, essa descrição é combinada com o horário da solicitação e com o relato fornecido pelo usuário para compor o contexto encaminhado ao modelo de linguagem.

O conjunto formado pelos prompts, pela localização e pelas modalidades recebidas é enviado à plataforma de distribuição de IAS, que realiza a comunicação com o modelo selecionado. As LLMS interpretam a ocorrência e retornam uma resposta com o serviço indicado, o número de contato e um resumo do contexto identificado.

Antes de devolver o resultado ao *frontend*, o *backend* verifica e organiza o conteúdo retornado, preservando somente os campos esperados pela aplicação. Após essa etapa, a resposta é enviada ao cliente, que apresenta ao usuário a classificação da ocorrência e as informações necessárias para o contato com o serviço indicado.

Esse fluxo mantém as integrações externas e o processamento da ocorrência concentrados no *backend*, reduzindo o acoplamento do *frontend* aos serviços utilizados. A sequência completa da comunicação entre o usuário, o *frontend*, e *backend* e as APIS externas pode ser observada na Figura 3.

Figura 3 – Fluxo de comunicação entre frontend, backend e APIs externas



Elaborado pelo próprio autor (2026).

4 Desenvolvimento

Neste capítulo são apresentados os detalhes de desenvolvimento do sistema Centro de Ajuda. São descritas as decisões de projeto, as tecnologias utilizadas, o funcionamento do aplicativo cliente e do servidor, a construção dos prompts enviados aos modelos de linguagem e o processo de avaliação por meio do *benchmark*. O objetivo deste capítulo é fornecer uma visão completa de como a solução foi implementada e como os componentes interagem para entregar ao usuário a recomendação do serviço de emergência mais adequado à situação descrita.

4.1 Abordagem Geral

O desenvolvimento do sistema foi dividido em duas camadas principais: a aplicação cliente, responsável pela interação com o usuário, e a aplicação servidora, encarregada do processamento das solicitações e da comunicação com os modelos de linguagem. Essa separação permite isolar as responsabilidades de cada componente, facilitando a manutenção, a evolução da arquitetura e a substituição de tecnologias sem impactar os demais módulos.

Nesta prova de conceito, o foco do desenvolvimento concentrou-se na implementação do *backend* e na validação do fluxo completo de processamento das solicitações de emergência. O cliente móvel foi representado por um protótipo funcional de interface elaborado no Figma, plataforma de *design* de interfaces, que foi utilizada para definir o fluxo de interação do usuário e demonstrar o comportamento esperado da aplicação. Dessa forma, embora as telas e a arquitetura do cliente tenham sido projetadas, sua implementação completa não fez parte do escopo desta etapa do trabalho.

4.2 Implementação da Aplicação Cliente

O desenvolvimento da aplicação cliente teve como principal objetivo fornecer uma interface simples, intuitiva e acessível para o registro de ocorrências de emergência.

Considerando que o usuário pode estar em uma situação de estresse ou risco, buscou-se reduzir ao máximo a quantidade de interações necessárias para realizar uma solicitação.

A aplicação foi projetada para dispositivos móveis e organizada em módulos responsáveis pela interface com o usuário, captura das diferentes formas de entrada e envio das informações ao servidor. Dessa forma, cada componente possui uma responsabilidade específica, facilitando tanto a manutenção quanto futuras expansões da aplicação.

4.2.1 Arquitetura do Cliente

A arquitetura da aplicação cliente foi organizada de acordo com o fluxo de interação do usuário, abrangendo a coleta das informações, a preparação da solicitação, a comunicação com o servidor e a apresentação do resultado.

Inicialmente, a tela de entrada permite que o usuário registre o relato por texto, áudio ou imagem. A aplicação também obtém as informações adicionais necessárias para contextualizar a ocorrência, como a localização geográfica e o horário da solicitação. Os dados coletados são encaminhados ao módulo de comunicação, responsável por organizá--los e enviá-los ao *backend*.

Durante o processamento da solicitação pelo servidor, a aplicação apresenta uma tela de espera, que informa ao usuário que a ocorrência está sendo analisada. Após o recebimento da resposta, o fluxo é direcionado à tela de resultado, na qual são apresentados o serviço de emergência recomendado, o número de telefone correspondente, o resumo do contexto identificado e a opção de iniciar a ligação.

Conforme a Figura 4, esses componentes foram organizados de maneira sequencial, acompanhando as etapas realizadas desde o registro da ocorrência até a apresentação da recomendação ao usuário.

Figura 4 – Arquitetura da aplicação cliente



Elaborado pelo próprio autor (2026).

4.2.2 Tecnologias da Aplicação Cliente

A interface da aplicação foi desenvolvida com tecnologias consolidadas no desenvolvimento *web*. O HTML5 foi utilizado na estruturação das telas, enquanto o CSS3 foi empregado na estilização dos componentes. O JavaScript foi responsável pelo gerenciamento das interações, pela captura dos dados, pelo controle da navegação entre as telas e pela comunicação com o servidor.

O *framework* Bootstrap foi utilizado para padronizar os elementos visuais e auxiliar na construção de uma interface responsiva, adequada às diferentes dimensões de tela dos dispositivos móveis.

A comunicação entre a aplicação cliente e o *backend* foi implementada por meio da Fetch API. Após a coleta das informações da ocorrência, o JavaScript organiza os dados e realiza uma requisição HTTP ao servidor. A resposta recebida é interpretada pelo cliente e utilizada para preencher a tela de resultado.

Implementação PWA

Com o objetivo de facilitar o acesso ao sistema em dispositivos móveis, a aplicação cliente foi implementada como uma PWA básica. Essa abordagem permite que o sistema seja executado diretamente pelo navegador e adicionado à tela inicial do dispositivo, oferecendo uma experiência semelhante à de uma aplicação instalada, sem exigir seu empacotamento como aplicativo nativo.

A implementação da PWA foi realizada por meio da inclusão de um arquivo de manifesto e de um *Service Worker*. O arquivo

`manifest.json`

fornece ao navegador informações de identificação e apresentação da aplicação, como nome, ícones, cores, endereço inicial e modo de exibição. Entre suas configurações, a propriedade display, definida como standalone, permite que a aplicação seja executada em uma janela independente, sem os elementos convencionais de navegação do navegador. Os ícones especificados no manifesto são utilizados na tela inicial do dispositivo e em outros elementos de identificação da aplicação.

Complementarmente, o *Service Worker* consiste em um *script* executado em segundo plano pelo navegador. Na aplicação desenvolvida, ele é responsável pelo armazenamento local dos arquivos essenciais da interface, possibilitando que esses recursos sejam recuperados sem a necessidade de serem baixados novamente a cada acesso. Em conjunto, o manifesto e o *Service Worker* fornecem as funcionalidades necessárias para que a aplicação *web* apresenta características semelhantes às de uma aplicação instalada no dispositivo.

4.2.3 Tela Inicial

A tela inicial da aplicação concentra todas as funcionalidades necessárias para o registro da ocorrência, sendo projetada em um design minimalista para minimizar a quantidade de interações exigidas do usuário em uma situação de emergência.

Conforme a Figura 5 apresenta, a interface adota um tema escuro com elementos de destaque em azul, priorizando contraste visual e facilidade de identificação das ações principais. Na parte superior da tela é apresentado o nome da aplicação, acompanhado de uma breve instrução orientando o usuário a descrever a ocorrência.

A região central da interface é composta por um campo destinado à descrição textual da emergência e por botões para captura de áudio e imagem, permitindo que

diferentes tipos de informação sejam enviados ao sistema. Na parte inferior encontra-se o botão “Pedir Ajuda”, responsável por iniciar o envio da solicitação ao servidor.

A organização desses elementos foi definida com o objetivo de reduzir a navegação entre telas e concentrar todos os recursos de entrada de dados em uma única interface, tornando o processo de solicitação mais rápido e intuitivo.

Figura 5 – Tela inicial do Centro de Ajuda



Elaborado pelo próprio autor (2026).

Captura de texto

A entrada textual foi implementada por meio de um elemento HTML do tipo

`<textarea>`

, no qual o usuário pode descrever a ocorrência em linguagem natural. O componente permite a inserção de textos com múltiplas linhas e limita o relato a 2.000 caracteres, evitando o envio de conteúdos excessivamente longos.

Código 1 – Campo de texto da ocorrência

occurrence.html

```
<textarea id="occurrence-text" name="text" rows="5" maxlength="2000"
placeholder="Descreva sua ocorrência..."></textarea>
```

Elaborado pelo próprio autor (2026).

O campo não foi definido como obrigatório porque a aplicação também permite o envio de solicitações compostas apenas por áudio ou imagem. A verificação da presença de pelo menos uma forma de entrada é realizada posteriormente, antes do envio da solicitação.

Quando o usuário confirma o envio, o conteúdo do campo é acessado pelo JavaScript por meio da Document Object Model (DOM). O método

`document.getElementById()`

localiza o elemento a partir de seu identificador, enquanto a propriedade value recupera o texto inserido. Em seguida, o método

`trim()`

remove espaços em branco desnecessários do início e do final do relato.

Código 2 – Captura de texto via DOM

capture.js

```
function captureText() {
  const textInput = document.getElementById("occurrence-text");
  const text = textInput.value.trim();
  return text || null;
}
```

Elaborado pelo próprio autor (2026).

Quando o campo está vazio ou contém apenas espaços, a função retorna

`null`

. Caso exista conteúdo válido, o texto é retornado para posteriormente ser incorporado à solicitação encaminhada ao servidor.

Captura de áudio

A captura de áudio foi implementada com a MediaDevices API. Quando o usuário inicia a gravação, o método

`navigator.mediaDevices.getUserMedia()`

solicita autorização para acessar o microfone do dispositivo. Caso a permissão seja concedida, o navegador fornece um fluxo de mídia, representado por um objeto MediaStream, que é disponibilizado para a aplicação.

Código 3 – Solicitação de acesso ao microfone

```
audio.js
1  let audioStream = null;
2  async function requestMicrophoneAccess() {
3    try {
4      audioStream = await navigator.mediaDevices.getUserMedia({ audio:
       true });
5      return audioStream;
6    } catch (error) {
7      console.error("Não foi possível acessar o microfone:", error);
8      return null;
9    }
10 }
```

Elaborado pelo próprio autor (2026).

Caso o usuário recuse a permissão ou o dispositivo não possua um microfone disponível, a função retorna

`null`

, permitindo que a aplicação informe o erro sem interromper as demais formas de entrada.

A gravação é realizada por meio da

`MediaRecorder`

API, que recebe o fluxo fornecido pelo microfone. Durante a gravação, os dados de áudio são disponibilizados em pequenos blocos, denominados *chunks*, e armazenados temporariamente em um vetor. Quando a gravação é encerrada, esses blocos são reunidos em um objeto *blob*, preservando o formato de áudio selecionado pelo navegador.

Código 4 – Gravação de áudio com MediaRecorder

recorder.js

```
let mediaRecorder = null;
let audioChunks = [];
let audioBlob = null;
async function startAudioRecording() {
  const stream = await requestMicrophoneAccess();
  if (!stream) return false;
  audioChunks = [];
  audioBlob = null;
  mediaRecorder = new MediaRecorder(stream);
  mediaRecorder.addEventListener("dataavailable", event => {
    if (event.data.size > 0) audioChunks.push(event.data);
  });
  mediaRecorder.addEventListener("stop", () => {
    audioBlob = new Blob(audioChunks, { type: mediaRecorder.mimeType });
    stream.getTracks().forEach(track => track.stop());
    audioStream = null;
  });
  mediaRecorder.start();
  return true;
}
function stopAudioRecording() {
  if (mediaRecorder?.state === "recording") mediaRecorder.stop();
}
```

Elaborado pelo próprio autor (2026).

O objeto

`audioBlob`

resultante permanece armazenado na aplicação até a confirmação do envio. Posteriormente, ele pode ser anexado diretamente à solicitação por meio da interface FormData, juntamente com as demais informações da ocorrência.

Captura de imagens

A captura de imagens foi implementada com os recursos disponibilizados pelo próprio navegador, sem a utilização de plugins ou integração direta com código nativo. Para isso, foi utilizado um campo de seleção de arquivos configurado para aceitar somente imagens.

Esse campo de seleção possui o atributo

`capture="environment"`

, que indica ao navegador a preferência pelo uso da câmera traseira do dispositivo. Entretanto, seu comportamento pode variar de acordo com o navegador e o sistema operacional, que também podem permitir a escolha de uma imagem já armazenada no dispositivo. Após a captura ou seleção, o navegador disponibiliza a imagem como um objeto do tipo

`File`

. O JavaScript obtém esse arquivo por meio da propriedade files do campo de entrada e verifica se o conteúdo selecionado corresponde a uma imagem.

A API

`FileReader`

é utilizada para gerar uma representação da imagem no formato

`Data URI`

, permitindo sua exibição na interface antes do envio. O arquivo original, contudo, permanece armazenado no objeto

`imageFile`

e será posteriormente anexado à solicitação por meio da interface

`BodyParser`

.

Código 5 – Captura de imagem

```
image.js
1   const imageInput = document.getElementById("image-input");
2   const imagePreview = document.getElementById("image-preview");
3   let imageFile = null;
4   imageInput.addEventListener("change", event => {
5     const selectedFile = event.target.files[0];
6     if (!selectedFile || !selectedFile.type.startsWith("image/")) return;
7     imageFile = selectedFile;
8     const reader = new FileReader();
9     reader.addEventListener("load", () => {
10      imagePreview.src = reader.result;
11      imagePreview.hidden = false;
12    });
13    reader.readAsDataURL(imageFile);
14  });
```

Elaborado pelo próprio autor (2026).

Envio da solicitação

Após a coleta das informações fornecidas pelo usuário, a aplicação reúne os dados da ocorrência em uma única solicitação destinada ao servidor. Dependendo das formas de entrada utilizadas, a solicitação pode conter um relato textual, um arquivo de áudio, uma imagem ou uma combinação desses conteúdos.

Os dados são organizados por meio da interface

`BodyParser`

, adequada ao envio conjunto de campos textuais e arquivos. O texto é adicionado somente quando estiver preenchido, enquanto o áudio e a imagem são anexados diretamente como objetos

`Blob`

e

`File`

, respectivamente.

A comunicação entre a aplicação cliente e o *backend* é realizada por meio da Fetch API, utilizando uma requisição HTTP do tipo POST. O objeto que representa a ocorrência é convertido para JSON com o método

`JSON.stringify()`

. Nesse caso, o cabeçalho

`Content-Type`

deve ser definido como

`application/json`

, informando ao servidor o formato do corpo da requisição.

Antes do envio, a aplicação verifica se pelo menos uma forma de entrada foi fornecida. Caso não exista texto, áudio ou imagem, a solicitação é interrompida e uma mensagem é apresentada ao usuário. Uma variável de controle também impede que uma nova requisição seja iniciada enquanto a anterior ainda estiver em processamento.

Código 6 – Envio da ocorrência ao servidor

submit.js

```
1  const helpForm = document.getElementById("help-form");
2  const submitButton = document.getElementById("submit-button");
3  let requestInProgress = false;
4  helpForm.addEventListener("submit", async event => {
5    event.preventDefault();
6    if (requestInProgress) return;
7    const text = captureText();
8    const hasAudio = audioBlob instanceof Blob;
9    const hasImage = imageFile instanceof File;
10   if (!text && !hasAudio && !hasImage) {
11     alert("Forneça um relato por texto, áudio ou imagem.");
12     return;
13   }
14   requestInProgress = true;
15   submitButton.disabled = true;
16   showScreen("waiting-screen");
17   try {
18     const location = await getCurrentLocation();
19     const occurrenceData = await buildOccurrenceData(text, location);
20     const response = await fetch("/help", {
21       method: "POST",
22       headers: { "Content-Type": "application/json" },
23       body: JSON.stringify(occurrenceData)
24     });
25     if (!response.ok) throw new Error(`Erro HTTP: ${response.status}`);
26     await handleServerResponse(response);
27   } catch (error) {
28     console.error("Erro ao enviar a solicitação:", error);
29     showResponseError("Não foi possível processar a solicitação. Tente
       novamente.");
30   } finally {
31     requestInProgress = false;
32     submitButton.disabled = false;
33   }
34 });
```

Elaborado pelo próprio autor (2026).

Após o envio da solicitação pela aplicação cliente, o *backend* recebe o corpo da requisição em formato JSON. O *middleware*

`bodyParser.json()`

é responsável por interpretar esse conteúdo e disponibilizá-lo no objeto

`req.body`

. O limite de tamanho aceito pelo *middleware* foi ampliado devido à presença dos arquivos de áudio e imagem codificados em Base64, representação que aumenta o volume de dados transmitidos.

Depois da interpretação do corpo da requisição, o endpoint extrai de

`req.body`

os dados enviados pela aplicação cliente. Esses dados incluem o relato textual, o áudio, a imagem, as coordenadas geográficas e o horário da ocorrência.

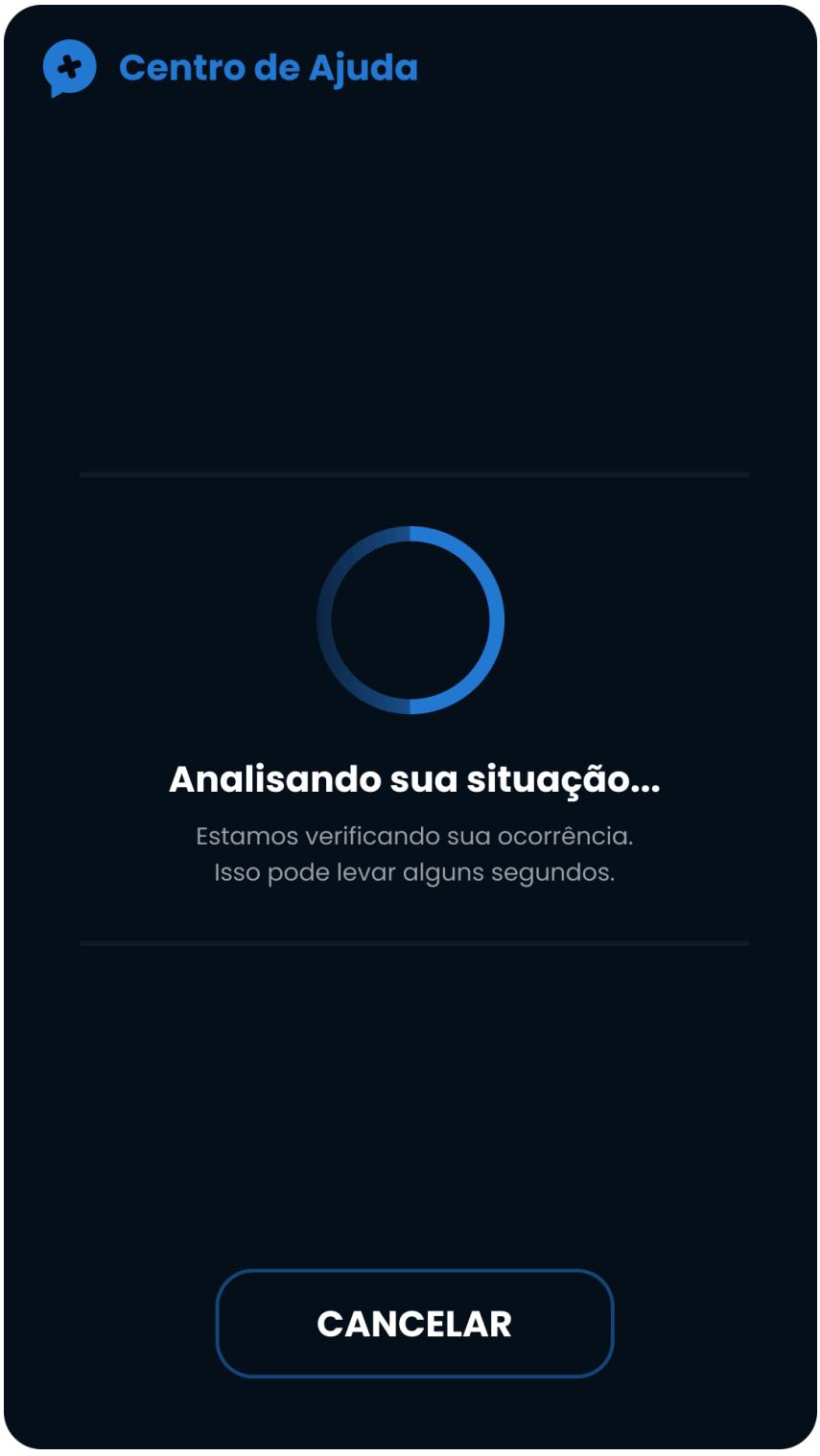
4.2.4 Tela de Espera

Após o envio da solicitação, a aplicação redireciona o usuário para uma tela intermediária, ilustrada em Figura 6. Essa tela tem como objetivo fornecer um retorno visual imediato de que a requisição foi recebida e está sendo processada pelo servidor, evitando que o usuário interprete a ausência de resposta como uma falha na aplicação.

A interface apresenta uma animação de carregamento acompanhada de uma mensagem informando que a ocorrência está sendo analisada. Durante esse período, novas interações relacionadas à solicitação são bloqueadas, impedindo o envio duplicado da mesma ocorrência e garantindo que apenas uma requisição permaneça em processamento.

A tela permanece visível até que o servidor retorne uma resposta contendo a recomendação do serviço de emergência ou informe a ocorrência de algum erro durante o processamento da solicitação.

Figura 6 – Tela de espera



Elaborado pelo próprio autor (2026).

4.2.5 Tela de Resposta

Após o processamento da solicitação, a aplicação apresenta ao usuário a tela de resposta, ilustrada na Figura 7. Essa interface foi projetada para que destaque as informações mais relevantes da recomendação gerada, permitindo que o usuário identifique rapidamente qual serviço de emergência deve ser acionado.

A tela exibe o nome do serviço recomendado, seu respectivo número telefônico e um resumo da ocorrência interpretada pelo sistema, oferecendo ao usuário uma justificativa simplificada da decisão tomada. Além disso, é apresentado um aviso informando que a recomendação foi produzida por um modelo de inteligência artificial e deve ser utilizada como um recurso de apoio, não substituindo a avaliação do próprio usuário diante da situação de emergência.

Figura 7 – Tela de resposta com recomendação



Elaborado pelo próprio autor (2026).

Recebimento da Resposta

O recebimento da resposta ocorre após a conclusão da requisição HTTP iniciada durante o envio da solicitação. Quando o servidor conclui o processamento, a aplicação interpreta o corpo da resposta por meio do método

`response.json()`

, que converte o conteúdo JSON recebido em um objeto

`JavaScript`

.

Uma resposta válida deve conter os campos

`service_name`

,

`phone_number`

e

`emergency_context`

, correspondentes, respectivamente, ao serviço de emergência recomendado, ao número de telefone e ao resumo da ocorrência identificado pelo modelo. Antes de apresentar o resultado, a aplicação verifica a existência e o tipo desses campos.

Caso a resposta seja válida, os elementos da interface são atualizados dinamicamente e a aplicação realiza a transição da tela de espera para a tela de resposta. Se o servidor retornar um erro ou se o conteúdo recebido não apresentar a estrutura esperada, a aplicação exibe uma mensagem informando que não foi possível concluir a análise e retorna o usuário à tela inicial.

Código 7 – Tratamento da resposta do servidor

handle.js

```
1  async function handleServerResponse(response) {
2    let result;
3    try {
4      result = await response.json();
5    } catch (error) {
6      showResponseError("O servidor retornou uma resposta inválida. Tente
       novamente.");
7      return;
8    }
9    const validResponse = response.ok && typeof result?.service_name ===
     "string" && typeof result?.phone_number === "string" && typeof
     result?.emergency_context === "string";
10   if (!validResponse) {
11     showResponseError(result?.error || "Não foi possível analisar a
       ocorrência. Tente novamente.");
12     return;
13   }
14   document.getElementById("service-name").textContent =
     result.service_name;
15   document.getElementById("phone-number").textContent =
     result.phone_number;
16   document.getElementById("emergency-context").textContent =
     result.emergency_context;
17   configureCallButton(result.phone_number);
18   showScreen("result-screen");
19 }
20 function showResponseError(message) {
21   const errorMessage = document.getElementById("request-error");
22   errorMessage.textContent = message;
23   errorMessage.hidden = false;
24   showScreen("initial-screen");
25 }
```

Elaborado pelo próprio autor (2026).

Redirecionamento para Ligação

Após a apresentação da recomendação, o usuário pode iniciar o contato com o serviço de emergência por meio do botão “Ligar Agora”. A funcionalidade utiliza o esquema de Uniform Resource Identifier (URI)

`tel`

, reconhecido pelos navegadores e pelos dispositivos móveis.

Ao acionar o botão, a aplicação abre o discador do dispositivo com o número recomendado previamente preenchido. A ligação não é realizada automaticamente, pois ainda depende da confirmação do usuário no aplicativo de chamadas.

O endereço do botão é definido somente após o recebimento e a validação da resposta do servidor. Antes de construir a URI, caracteres que não representam dígitos são removidos do número recebido.

Código 8 – Redirecionamento para ligação

```
call.js
1  function configureCallButton(phoneNumber) {
2    const callButton = document.getElementById("call-button");
3    const sanitizedNumber = phoneNumber.replace(/\D/g, "");
4    if (!sanitizedNumber) {
5      callButton.hidden = true;
6      callButton.removeAttribute("href");
7      return;
8    }
9    callButton.href = `tel:${sanitizedNumber}`;
10   callButton.hidden = false;
11 }
```

Elaborado pelo próprio autor (2026).

Essa abordagem elimina a necessidade de memorizar ou digitar manualmente o número recomendado, reduzindo o tempo necessário para acessar o serviço e a possibilidade de erros de discagem. Após o encerramento da ligação, o usuário pode retornar à aplicação e iniciar uma nova solicitação, caso seja necessário.

4.3 Implementação do Servidor

O servidor foi estruturado de forma modular, separando a inicialização da aplicação, o gerenciamento das requisições HTTP, o processamento das ocorrências, a comunicação com serviços externos e as funções auxiliares. Essa organização permite que cada parte do sistema seja modificada de maneira relativamente independente,

reduzindo a concentração da lógica em um único arquivo e facilitando a identificação das responsabilidades de cada componente.

A inicialização do sistema ocorre no arquivo principal da aplicação, responsável por carregar o módulo HTTP e iniciar o servidor na porta 27020. O serviço é configurado para aceitar conexões em todas as interfaces de rede disponíveis, por meio do endereço 0.0.0.0, possibilitando o acesso ao *backend* a partir de outros dispositivos conectados à rede ou de uma infraestrutura remota.

A camada de comunicação HTTP foi implementada com o *framework* Express. Nela são configurados os *middlewares* responsáveis pela interpretação dos corpos das requisições nos formatos JSON e URL encoded. Também foi habilitado o Cross-Origin Resource Sharing (CORS), permitindo que aplicações executadas em origens distintas realizem requisições ao servidor. Essa configuração é necessária porque a aplicação cliente e o *backend* podem ser executados em dispositivos ou endereços diferentes.

Código 9 – Inicialização do servidor

main.js

```
1 const { startExpress } = require("./source/express/express");
2 const { print } = require("./source/scripts/tools");
3
4 async function main() {
5     console.clear();
6     print("[Setup] Main module: Starting core systems: Please, wait...");
7     await startExpress(27020);
8 }
```

Elaborado pelo próprio autor (2026).

Código 10 – Configuração do Express

express.js

```
1  const express = require("express");
2  const cors = require("cors");
3  const bodyParser = require("body-parser");
4
5  const app = express();
6  app.use(bodyParser.json());
7
8  app.use(bodyParser.urlencoded({ extended: true }));
9
10 app.use(cors({
11     origin: "*",
12     methods: ["GET", "POST", "OPTIONS"],
13     allowedHeaders: ["Content-Type", "Authorization"],
14     credentials: false,
15     maxAge: 86400
16 }));
17
18 async function startExpress(port) {
19     setupEvents();
20     return new Promise((resolve, reject) => {
21         app.listen(port, "0.0.0.0", error => {
22             if (error) return reject(error);
23             print(`[Setup] Express: Listening on: ${port}`);
24             resolve();
25         });
26     });
27 }
28
29 module.exports = { startExpress };
```

Elaborado pelo próprio autor (2026).

Os endpoints da aplicação são carregados automaticamente a partir do diretório destinado às rotas do servidor. Durante a inicialização, o sistema percorre os arquivos JavaScript presentes nesse diretório, importa seus respectivos módulos e registra uma rota HTTP do tipo

`POST`

com o mesmo nome de cada arquivo. Dessa forma, o arquivo

`help.js`

é disponibilizado como o *endpoint*

`/help`

, responsável pelo recebimento e pelo processamento das solicitações de emergência.

Antes de encaminhar a requisição ao endpoint correspondente, a camada Express registra o endereço de origem da conexão, verifica a existência do corpo da requisição e define o tipo de conteúdo da resposta como JSON com codificação UTF-8. Após essas verificações, o controle é transferido ao módulo responsável pela rota solicitada.

Código 11 – Tratamento das solicitações ao servidor

handleendpoint.js

```
1  const path = require("path");
2  const fs = require("fs");
3
4  const endpointHandlers = {};
5  const endpointHandlersDir = path.join(__dirname, "./endpoints");
6
7  fs.readdirSync(endpointHandlersDir).forEach(file => {
8      if (!file.endsWith(".js")) return;
9      const endpointName = path.basename(file, ".js");
10     endpointHandlers[endpointName] =
       require(path.join(endpointHandlersDir,
11     file));
12 });
13
14 function setupEvents() {
15     for (const endpointName in endpointHandlers) {
16         print(`[Setup] Express: Event created: ${endpointName}`);
17         app.post(`/${endpointName}`, (req, res) => {
18             handleRequest(req, res, endpointName);
19         });
20     }
21 }
22
23 function handleRequest(req, res, endpointName) {
24     const address =
25         req.headers["x-forwarded-for"] ||
26         req.connection?.remoteAddress ||
27         "Debug";
28
29     print(`[Request] Express: Web request by: ${address}`);
30
31     if (!req.body) return res.status(400).json({ error: "Missing
       body" });
32     res.setHeader("Content-Type", "application/json; charset=utf-8");
33     print(`[Log] Express: Handling request: ${endpointName}`);
34     const handler = endpointHandlers[endpointName];
35     if (handler) return handler(req, res);
36 }
```

Elaborado pelo próprio autor (2026).

O *endpoint*

`/help`

concentra a coordenação do fluxo de atendimento. Esse módulo recebe os dados enviados pelo cliente, valida a presença das coordenadas geográficas, obtém a localização aproximada da ocorrência, organiza as informações em uma estrutura de contexto, constrói os *prompts* e solicita o processamento ao modelo de linguagem. Ao final, o resultado estruturado é devolvido à aplicação cliente por meio da resposta HTTP.

A comunicação com o serviço de geocodificação foi isolada em um módulo responsável pelas requisições de rede. Esse componente realiza a consulta à API Google Reverse Geocoding, utilizada para converter as coordenadas geográficas da ocorrência em informações textuais de localização. Esse processo permite complementar os dados de latitude e longitude com elementos de endereço mais compreensíveis e relevantes para a análise da ocorrência.

Para realizar a consulta, o servidor insere as coordenadas recebidas no endereço da API e envia uma requisição HTTP. Após o recebimento da resposta em formato JSON, os dados são encaminhados à função

`extractLocation()`

, responsável por percorrer os componentes do endereço e identificar os campos utilizados pelo sistema, como rua, bairro, cidade, estado, país e código postal. Ao final do processamento, essas informações são organizadas em um objeto e posteriormente incorporadas ao contexto da ocorrência.

Código 12 – Geocodificação reversa

geocode.js

```
async function getLocation(lat, lon) {
  const url = "https://maps.googleapis.com/maps/api/geocode/json" + `?latlng=${lat},${lon}` + `&key=${process.env.GOOGLEKEY}`;
  const response = await fetch(url);
  const data = await response.json();
  return extractLocation(data);
}
function extractLocation(data) {
  const components = data.results[0].address_components;
  let street = null, neighborhood = null, city = null, state = null, country = null, zip = null;
  for (const component of components) {
    const types = component.types;
    if (types.includes("route")) street = component.long_name;
    if (types.includes("sublocality") || types.includes("sublocality_level_1") || types.includes("neighborhood")) neighborhood = component.long_name;
    if (types.includes("administrative_area_level_2")) city = component.long_name;
    if (types.includes("administrative_area_level_1")) state = component.long_name;
    if (types.includes("country")) country = component.long_name;
    if (types.includes("postal_code")) zip = component.long_name;
  }
  return { street, neighborhood, city, state, country, zip };
}
```

Elaborado pelo próprio autor (2026).

Na comunicação com a plataforma de distribuição de modelos, o servidor organiza a requisição de acordo com o formato de mensagens utilizado pela API. São incluídas uma mensagem de sistema, uma mensagem de contextualização e uma mensagem contendo os dados fornecidos pelo usuário. Quando arquivos de imagem ou áudio estão presentes, eles são lidos do diretório de recursos, convertidos para Base64 e incorporados ao conteúdo multimodal da requisição.

A resposta retornada pelo modelo também é tratada nessa camada. Além do conteúdo gerado, o módulo registra informações relacionadas à execução, como tempo

de resposta, quantidade de tokens de entrada e saída e custo informado pela API. O conteúdo produzido pelo modelo é encaminhado à função

`extraction()`

, responsável por remover eventuais marcações de bloco de código e convertê-lo em um objeto

`JSON`

válido.

As funções utilitárias também foram reunidas em um módulo próprio. Entre elas estão a criação e localização de arquivos, o registro persistente de *logs*, a formatação das mensagens exibidas no terminal, a conversão de arquivos para Base64, a extração de objetos

`JSON`

e a interpretação dos dados retornados pelo serviço de geocodificação.

A função

`fileToBase64()`

realiza a leitura síncrona do arquivo e converte seu conteúdo para uma sequência Base64. A função

`extractJson()`

, por sua vez, tenta interpretar diretamente a resposta como

`JSON`

. Caso existam textos ou marcações adicionais, a função localiza o primeiro objeto delimitado por chaves e realiza uma nova tentativa de conversão.

Código 13 – Conversão de arquivos para Base64

tobase64.js

```
1  function fileToBase64(filepath) {
2      print("[Log] Convert: Creating base64 file");
3      const completePath = path.join(__dirname, "..", "assets", filepath );
4      return fs.readFileSync(completePath).toString("base64");
5  }
6
7  function extractJson(text) {
8      if (!text) return null;
9
10     const cleaned = String(text).replace(/```json/gi, "").replace(/```/g,
       "").trim();
11
12     try {
13         return JSON.parse(cleaned);
14     } catch {}
15
16     const first = cleaned.indexOf("{");
17     const last = cleaned.lastIndexOf("}");
18
19     if (first === -1 || last === -1 || last <= first) return null;
20
21     try {
22         return JSON.parse(cleaned.slice(first, last + 1));
23     } catch { return null; }
24 }
```

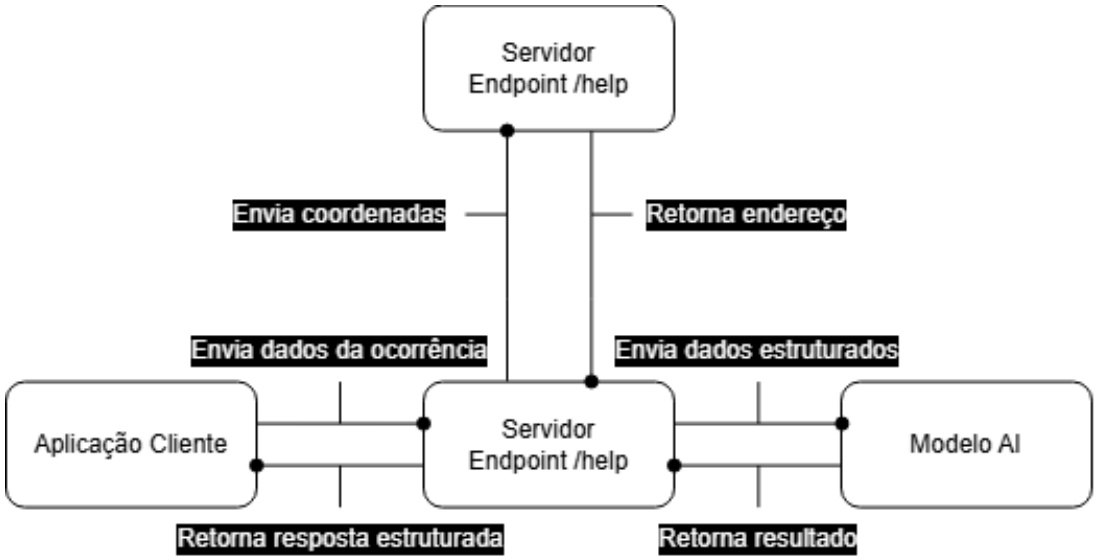
Elaborado pelo próprio autor (2026).

A Figura 8 apresenta a relação entre os principais componentes da arquitetura. A aplicação cliente envia a solicitação ao *endpoint*

`/help`

, que coordena o processamento e utiliza os módulos de prompt, geocodificação e comunicação com o modelo. Após a interpretação e validação da resposta, o resultado estruturado é devolvido ao cliente.

Figura 8 – Arquitetura geral do sistema



Elaborado pelo próprio autor (2026).

A separação adotada também evita que as credenciais de acesso sejam expostas na aplicação cliente, uma vez que as chaves utilizadas pelo Google Maps e pelo modelo IA permanecem armazenadas nas variáveis de ambiente do servidor. Além disso, a centralização das integrações no *backend* permite alterar o modelo de linguagem, os *prompts* ou os serviços externos sem exigir modificações diretas na interface da aplicação.

4.4 Construção do Prompt

Antes de realizar a comunicação com o modelo de linguagem, o servidor constrói o conjunto de mensagens que será encaminhado à API do Open Router. Esse conjunto, denominado *prompt*, define tanto o comportamento esperado do modelo quanto às informações específicas da ocorrência que deverão ser consideradas durante a classificação.

Em vez de encaminhar apenas o relato fornecido pelo usuário, o sistema complementa a solicitação com informações contextuais obtidas automaticamente durante o processamento, como localização geográfica e horário da ocorrência. Essa estratégia reduz ambiguidades e fornece ao modelo mais elementos para identificar o serviço de emergência adequado.

Na implementação desenvolvida, o prompt é organizado em três componentes: *System Prompt*, *Assistant Prompt* e mensagem do usuário. Cada componente é enviado com um papel diferente na estrutura de mensagens utilizada pela API.

O *System Prompt* estabelece a função do modelo, o processo de análise, as restrições da classificação e o formato obrigatório da resposta. Entre as regras definidas estão a consideração conjunta das modalidades de entrada, a escolha de apenas um serviço, a priorização de órgãos oficiais, a proibição de inventar informações e a exigência de uma resposta em

`JSON`

.

Código 14 – System Prompt

systemprompt.js

```
const systemprompt = `
# Função
Você é um classificador de ocorrências.
Analise todas as informações disponíveis e determine
qual serviço deve ser contatado.

# Processo
1. Analise todas as evidências disponíveis.
2. Se não for relacionado a um número oficial de
   emergência, pesquise serviços compatíveis.
3. Considere localização, horário e área de atendimento.
4. Escolha o serviço mais apropriado.

# Regras
- Considere texto, áudio, imagem, localização e horário
  em conjunto.
- Priorize serviços oficiais.
- Escolha apenas um serviço.
- Use pesquisa apenas quando necessário.
- Não forneça diagnósticos, orientações, tratamentos
  ou primeiros socorros.
- Não invente telefones, serviços, unidades, endereços,
  horários ou qualquer informação não verificada.
- Se não for possível identificar um serviço local,
  utilize o serviço oficial mais apropriado.

# Saída
Não use Markdown.
Não use bloco de código.
Não explique.
Não comente.
Retorne somente JSON válido:
{
  "service_name": "Nome do serviço",
  "phone_number": "Telefone",
  "emergency_context": "Resumo objetivo da ocorrência"
}
`;
```

Elaborado pelo próprio autor (2026).

O segundo componente corresponde ao *Assistant Prompt*, gerado dinamicamente para cada requisição. Diferentemente do *System Prompt*, esse componente apresenta informações específicas da ocorrência, como horário, rua, bairro, cidade, estado, país e código postal.

Essas informações são obtidas a partir do objeto de contexto construído pelo *endpoint*

`/help`

. Dessa forma, cada solicitação recebe sua própria contextualização antes de ser encaminhada ao modelo.

Código 15 – Assistant Prompt contextual

assistantprompt.js

```
function assistantprompt(context) {
    return `
# Informações
Horário/Dia Atual: ${context.emergency.time}
Rua: ${context.location.street}
Bairro: ${context.location.neighborhood}
Cidade: ${context.location.city}
Estado: ${context.location.state}
País: ${context.location.country}
CEP: ${context.location.zip}
`;
}
```

Elaborado pelo próprio autor (2026).

O terceiro componente corresponde à mensagem do usuário. Ela reúne o conteúdo efetivamente fornecido pela aplicação cliente e pode conter texto, imagem, áudio ou uma combinação dessas modalidades.

O relato textual é inserido como conteúdo do tipo text. As imagens são convertidas para Base64 e incorporadas como

`Data URI`

no campo

`image_url`

. Os arquivos de áudio também são convertidos para Base64 e adicionados como conteúdo do tipo

`input_audio`

, acompanhado da identificação de seu formato.

Código 16 – Construção do conteúdo multimodal

buildcontent.js

```
const content = [];

if (user) {
    content.push({
        type: "text",
        text: user
    });
}

if (image) {
    const imageFormat =
        image.split(".").pop().toLowerCase();

    content.push({
        type: "image_url",
        image_url: {
            url:
                `` +
                fileToBase64(image)
        }
    });
}

if (audio) {
    content.push({
        type: "input_audio",
        input_audio: {
            data: fileToBase64(audio),
            format: "ogg"
        }
    });
}
```

Elaborado pelo próprio autor (2026).

Após a preparação dos três componentes, as mensagens são organizadas conforme os papéis esperados pela API. As regras de classificação são enviadas com o papel *system*, a contextualização da ocorrência com o papel *assistant* e os dados fornecidos pelo usuário com o papel *user*.

4.4.1 Contextualização da Ocorrência

Uma característica importante da implementação é que o modelo de linguagem não recebe apenas o relato informado pelo usuário. Antes da comunicação com a API, o servidor realiza uma etapa de enriquecimento da solicitação, incorporando informações adicionais obtidas durante o processamento da requisição.

Entre esses dados encontram-se a latitude e longitude fornecidas pelo dispositivo móvel, o endereço aproximado obtido por meio do Google Reverse Geocoding e o horário em que a solicitação foi realizada. Essas informações são reunidas no objeto de contexto da aplicação e utilizadas para compor o *Assistant Prompt*.

A inclusão desse contexto permite reduzir ambiguidades presentes em descrições curtas ou incompletas. Por exemplo, um usuário pode informar apenas “há um incêndio próximo de casa”. Embora essa descrição indique a natureza da ocorrência, ela não fornece qualquer informação sobre sua localização. Ao incorporar automaticamente o endereço aproximado da ocorrência, o servidor fornece ao modelo elementos adicionais para interpretar a situação de maneira mais consistente e selecionar o serviço de emergência adequado.

Além das informações geográficas, o sistema também encaminha os conteúdos multimodais presentes na solicitação. Quando imagens ou gravações de áudio são fornecidas, esses arquivos são preparados pelo servidor e enviados juntamente com as mensagens textuais, permitindo que modelos compatíveis realizem uma análise conjunta das diferentes modalidades de entrada.

4.4.2 Formato de Saída (JSON)

Após concluir a análise da ocorrência, o modelo deve produzir uma resposta estruturada em formato

`JSON`

. A utilização de um formato padronizado elimina a necessidade de interpretar respostas em linguagem natural e reduz a possibilidade de ambiguidades durante a comunicação entre o servidor e a aplicação cliente.

O *System Prompt* instrui explicitamente o modelo a responder exclusivamente com um objeto

`JSON`

válido, sem comentários, blocos de código ou qualquer texto adicional. Essa restrição simplifica o processamento realizado pelo servidor, que pode converter diretamente a resposta recebida em um objeto da aplicação.

Na implementação atual, o

`JSON`

contém três campos obrigatórios:

`service_name`

, responsável por identificar o órgão de atendimento recomendado;

`phone_number`

, contendo o número telefônico correspondente; e

`emergency_context`

, que apresenta um resumo objetivo da interpretação realizada pelo modelo.

Após o recebimento da resposta, o servidor verifica se esses campos estão presentes antes de encaminhar o resultado à aplicação cliente. Caso a estrutura não esteja de acordo com o formato esperado, a resposta é considerada inválida e uma mensagem de erro é retornada ao cliente. Essa validação reduz falhas de integração e garante que

apenas respostas compatíveis com a arquitetura da aplicação sejam utilizadas durante o atendimento das ocorrências.

4.5 Integração com Modelos de Linguagem

Para realizar a integração com Modelos de Linguagem, foi utilizado o OpenRouter, uma plataforma que centraliza o acesso a modelos de IA desenvolvidos e disponibilizados por diferentes provedores. Por meio de uma interface de programação unificada, a plataforma atua como intermediária entre a aplicação e os modelos selecionados, padronizando o formato das requisições e respostas.

Essa abordagem simplifica a implementação do servidor, uma vez que as requisições seguem uma estrutura comum independentemente do modelo selecionado. Além disso, a interface unificada facilita a substituição ou inclusão de novos modelos durante o desenvolvimento e a realização dos experimentos.

A comunicação com os modelos é inteiramente realizada no servidor. Dessa forma, as credenciais de acesso à API permanecem protegidas e não são expostas na aplicação cliente, reduzindo os riscos de utilização indevida e permitindo maior controle sobre os modelos empregados pelo sistema.

4.5.1 Comunicação com o OpenRouter

A comunicação com o OpenRouter é realizada por meio de uma requisição HTTP do tipo POST para o endpoint de modelos conversacionais disponibilizado pela plataforma. O corpo da requisição é transmitido em formato JSON e segue a estrutura de mensagens definida pela API.

Após a construção dos prompts e a preparação dos conteúdos multimodais, o servidor organiza a requisição com o identificador do modelo selecionado, as mensagens correspondentes aos papéis system, assistant e user e a indicação de que a resposta não deve ser transmitida de forma fragmentada. A autenticação é realizada por meio de uma chave de acesso armazenada nas variáveis de ambiente do servidor.

Código 17 – Envio da requisição ao OpenRouter

requestopenrouter.js

```
const openrouterURL =
    "https://openrouter.ai/api/v1/chat/completions";

const response = await fetch(openrouterURL, {
    method: "POST",
    headers: {
        Authorization:
            `Bearer ${dotenv.OPENROUTERAPIKEY}`,
        "Content-Type": "application/json"
    },
    body: JSON.stringify(body)
});
```

Elaborado pelo próprio autor (2026).

A chave utilizada na autenticação não é incorporada diretamente ao código-fonte. Ela é obtida por meio do pacote dotenv, que carrega as variáveis definidas no ambiente de execução do servidor. Essa abordagem evita que a credencial seja exposta na aplicação cliente ou registrada diretamente nos arquivos da aplicação.

Após o processamento realizado pelo modelo, o OpenRouter retorna um objeto `JSON` contendo o conteúdo gerado e informações relacionadas à execução da requisição. O servidor registra o tempo decorrido entre o envio e o recebimento da resposta, além da quantidade de tokens de entrada, saída e total, e do custo informado pela API.

Essas informações são armazenadas juntamente com o resultado e utilizadas durante o *benchmark* para comparar o comportamento dos modelos avaliados. O tempo registrado representa a duração total observada pelo servidor durante a comunicação, incluindo o processamento remoto e a transmissão da resposta.

Código 18 – Registro da resposta e das métricas da requisição

openrouteres.js

```
const startedAt = Date.now();

const response = await fetch(openrouterURL, {
    method: "POST",
    headers: {
        Authorization:
            `Bearer ${dotenv.OPENROUTERAPIKEY}`,
        "Content-Type": "application/json"
    },
    body: JSON.stringify(body)
});

const data = await response.json();
const responseTime = Date.now() - startedAt;

if (!response.ok) {
    return {
        content: null,
        code: data.error?.code,
        error: data.error?.message,
        cost: 0,
        response_time_ms: responseTime,
        prompt_tokens: 0,
        completion_tokens: 0,
        total_tokens: 0
    };
}

const result = {
    content: extractJson(
        data.choices?.[0]?.message?.content
    ),
    cost: data.usage?.cost ?? 0,
    response_time_ms: responseTime,
    prompt_tokens:
        data.usage?.prompt_tokens ?? 0,
    completion_tokens:
        data.usage?.completion_tokens ?? 0,
    total_tokens:
        data.usage?.total_tokens ?? 0
};
```

Elaborado pelo próprio autor (2026).

Antes de encaminhar a recomendação à aplicação cliente, o servidor verifica se o conteúdo retornado foi convertido em um objeto

`JSON`

e se apresenta os três campos obrigatórios:

`service_name`

,

`phone_number`

e

`emergency_context`

.

Caso a resposta não possua a estrutura esperada, o resultado é considerado inválido e o servidor retorna uma mensagem de erro. Quando a validação é concluída com sucesso, somente os campos necessários à tela de resposta são encaminhados ao cliente.

Código 19 – Validação da estrutura da resposta

validateopenrouter.js

```
1  const recommendation = result?.content;
2
3  const validResult =
4      typeof recommendation?.service_name === "string" &&
5      typeof recommendation?.phone_number === "string" &&
6      typeof recommendation?.emergency_context === "string";
7
8  if (!validResult) {
9      return res.status(500).json({
10         success: false,
11         reason: "Invalid model response"
12     });
13 }
14
15 return res.status(200).json({
16     success: true,
17     service_name:
18         recommendation.service_name,
19     phone_number:
20         recommendation.phone_number,
21     emergency_context:
22         recommendation.emergency_context
23 });
24
```

Elaborado pelo próprio autor (2026).

5 Análise dos Resultados

Com o desenvolvimento da prova de conceito, tornou-se necessário avaliar como diferentes modelos de linguagem se comportam quando integrados à arquitetura proposta. Essa etapa teve como finalidade verificar não apenas a capacidade dos modelos de produzir respostas, mas também sua adequação às necessidades do Centro de Ajuda, especialmente diante das diferentes formas de entrada previstas pelo sistema.

A avaliação considera aspectos relacionados à interpretação de texto, áudio e imagem, ao cumprimento do formato de resposta estabelecido, à adequação do serviço recomendado e ao comportamento apresentado durante a comunicação com a API. Também foram observados indicadores como tempo de resposta, custo e quantidade de *tokens* processados, permitindo analisar tanto os resultados produzidos quanto às condições técnicas de utilização de cada modelo.

Para possibilitar essa comparação, foi elaborado um *benchmark* composto por cenários padronizados, representativos das modalidades de entrada suportadas pela aplicação. Os mesmos casos foram submetidos aos modelos selecionados por meio da infraestrutura implementada no servidor, proporcionando condições semelhantes de execução e permitindo identificar diferenças de compatibilidade, funcionamento e desempenho entre eles.

5.1 Modelos Testados

Os modelos utilizados no *benchmark* correspondem aos mesmos modelos previamente selecionados durante a etapa metodológica deste trabalho. A escolha considerou critérios relacionados à disponibilidade comercial, integração por API, suporte ao processamento multimodal e possibilidade de utilização em infraestrutura em nuvem. O *benchmark* tem como objetivo verificar, na prática, como esses modelos se comportam quando submetidos ao fluxo completo implementado na prova de conceito, complementando a análise teórica apresentada anteriormente.

Foram avaliados os modelos: OpenAI GPT-5.5; Anthropic Claude Opus 4.8; Meta Llama 4 Maverick; Alibaba Qwen 3.6 Plus; Google Gemini 3.5 Flash; NVIDIA Nemotron 3 Nano Omni 30B A3B Reasoning; Cohere North Mini Code.

Todos os modelos foram acessados por meio da plataforma OpenRouter, permitindo que as solicitações utilizassem uma interface padronizada independentemente do provedor responsável pelo modelo.

5.2 Cenários Testados

A avaliação utilizou quatro cenários de emergência elaborados para representar diferentes combinações de modalidades de entrada suportadas pela aplicação. A tabela abaixo apresenta uma síntese desses cenários, indicando a situação representada e as modalidades fornecidas em cada teste. Em seguida, cada cenário é descrito individualmente.

Tabela 2 – Cenários testados

Cenário	**Entrada**
Primeiro	Texto
Segundo	Texto e Imagem
Terceiro	Áudio
Quarto	Áudio e Imagem

Elaborado pelo próprio autor (2026).

O primeiro cenário utiliza apenas descrição textual, simulando uma situação na qual um veículo apresenta defeito no acostamento de uma rodovia durante o período noturno.

O segundo cenário combina texto e imagem, representando uma árvore caída sobre fios elétricos após uma tempestade, permitindo avaliar a utilização conjunta de informações textuais e visuais.

O terceiro cenário utiliza apenas áudio, simulando o relato de um atropelamento de animal, permitindo verificar tanto a compatibilidade dos modelos com processamento sonoro quanto sua capacidade de interpretar o conteúdo da gravação.

O quarto cenário combina áudio e imagem, representando uma residência sendo invadida pela água durante fortes chuvas. Esse cenário foi desenvolvido para avaliar a capacidade dos modelos de integrar simultaneamente informações provenientes de diferentes modalidades durante a classificação da ocorrência. Todos os cenários também incluem coordenadas geográficas e horário da ocorrência, permitindo que o servidor complemente automaticamente o contexto antes da comunicação com o modelo de linguagem.

5.3 Coleta de Resultados

Ao término de cada execução, o *benchmark* registra integralmente os dados utilizados durante a solicitação e a resposta produzida pelo modelo. Inicialmente essas informações são armazenadas no arquivo

`benchmark.txt`

, preservando tanto os dados de entrada quanto o objeto retornado pela integração com o OpenRouter.

Além da resposta produzida pelo modelo, também são registrados indicadores técnicos fornecidos pela API, incluindo o tempo de resposta, o custo estimado da requisição e as quantidades de *tokens* de entrada, de saída e total. Quando a execução não é concluída com sucesso, o código e a mensagem de erro retornados pelo provedor também são armazenados. A tabela abaixo apresenta, como exemplo, um dos dados gerados e registrados em uma das execuções realizadas com os modelos de linguagem.

Tabela 3 – Exemplo de métricas por modelo

Parâmetro	**GPT-5.5**	**Claude**	**Llama**	**Qwen**	**Gemini**	**Nemo-tron**	**North**
cost	0.107	0.114	0.006	0.011	0.012	0.005	0.005
response_time_ms	44059	11028	2634	47734	10398	3910	6445
prompt_tokens	15370	19821	3658	3837	524	3982	3655
completion_tokens	972	227	48	2496	1336	351	497

Elaborado pelo próprio autor (2026).

5.4 Comparação por Critérios

Nesta primeira análise, os modelos foram avaliados de forma isolada, considerando exclusivamente as modalidades de entrada processadas diretamente por cada modelo em uma única requisição. Portanto, não foram utilizados modelos auxiliares para transcrição de áudio, interpretação de imagens ou conversão prévia dos conteúdos. Essa abordagem permitiu identificar as capacidades multimodais nativas de cada modelo e verificar sua compatibilidade direta com os requisitos da aplicação.

Os modelos foram comparados segundo os principais critérios adotados durante o *benchmark*. A tabela abaixo apresenta uma síntese do comportamento observado nos quatro cenários, permitindo visualizar a compatibilidade de cada modelo com as modalidades de entrada e a qualidade geral das respostas produzidas.

Os critérios consideram tanto aspectos técnicos da integração quanto a adequação das recomendações. O processamento de uma modalidade foi considerado bem-sucedido quando a requisição foi aceita pelo provedor e produziu uma resposta coerente com o conteúdo enviado. Portanto, a simples aceitação de um arquivo não foi considerada suficiente quando o modelo não demonstrou compreender seu conteúdo.

O critério “JSON válido” indica se as respostas apresentaram os três campos obrigatórios (

`service_name`

,

`phone_number`

e

`emergency_context`

). A classificação “Completo” foi atribuída ao modelo que retornou a estrutura esperada nos quatro cenários, enquanto “Parcial” indica que apenas parte das execuções produziu o JSON exigido. A “Adequação da resposta” corresponde à avaliação quali-

tativa da relação entre a ocorrência apresentada, o serviço recomendado, o telefone informado e o resumo produzido.

Tabela 4 – Comparação por critérios — modelo isolado

Critério	**GPT**	**Claude**	**Llama**	**Qwen**	**Gemini**	**Nemotron**	**North**
Texto	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso
Texto+Imagem	Sucesso	Falha	Sucesso	Sucesso	Sucesso	Sucesso	Falha
Áudio	Falha	Falha	Falha	Falha	Sucesso	Falha	Falha
Áudio+Imagem	Falha	Falha	Falha	Falha	Sucesso	Falha	Falha
JSON	Parcial	Parcial	Parcial	Parcial	Completo	Parcial	Parcial
Resposta	Boa	Boa	Boa	Boa	Excelente	Regular	Boa

Elaborado pelo próprio autor (2026).

Observa-se que apenas o Gemini 3.5 Flash atendeu integralmente aos requisitos da aplicação, processando todas as modalidades de entrada e retornando respostas válidas em todos os cenários. GPT-5.5, Llama 4 Maverick e Qwen 3.6 Plus apresentaram comportamento semelhante, produzindo respostas satisfatórias para texto e imagem, porém sem compatibilidade com entradas contendo áudio. O Claude Opus 4.8 demonstrou limitações adicionais durante o processamento de imagens, enquanto o North Mini Code restringiu-se ao processamento textual. O Nemotron destacou-se por aceitar parte das entradas de áudio, entretanto a interpretação produzida mostrou-se insuficiente para caracterizar um processamento multimodal efetivo.

5.4.1 Segunda Análise

Os resultados obtidos na primeira análise não foram considerados satisfatórios para uma comparação definitiva entre as alternativas avaliadas. Ao utilizar cada modelo isoladamente, a análise ficou restrita às modalidades aceitas diretamente pelo modelo principal. Consequentemente, modelos com bom desempenho na classificação das ocorrências foram prejudicados por não processarem determinados tipos de entrada,

principalmente áudio, mesmo quando seus respectivos ecossistemas disponibilizam modelos auxiliares capazes de realizar esse processamento.

Essa limitação também tornou a comparação desigual. Enquanto alguns modelos possuem suporte multimodal nativo e conseguem interpretar texto, áudio e imagem em uma única requisição, outros foram projetados para atuar em conjunto com modelos especializados. Nesses casos, um modelo pode transcrever o áudio ou interpretar a imagem, enquanto outro recebe o conteúdo processado e realiza a classificação final da ocorrência.

Diante disso, foi realizada uma segunda análise considerando o ecossistema de modelos disponível no OpenRouter. Nessa abordagem, diferentes modelos de um mesmo ecossistema podem ser executados sequencialmente. Um arquivo de áudio, por exemplo, pode ser inicialmente enviado a um modelo especializado, responsável por produzir sua transcrição ou interpretação textual. Em seguida, esse conteúdo é encaminhado ao modelo principal juntamente com as demais informações da ocorrência.

A segunda análise busca, portanto, avaliar se as limitações observadas anteriormente podem ser compensadas por meio da composição de modelos. Além da compatibilidade com as modalidades de entrada, essa abordagem permite analisar os possíveis impactos da utilização de múltiplas etapas sobre o tempo de resposta, o custo e a qualidade da recomendação produzida.

A tabela abaixo apresenta a viabilidade das combinações identificadas no catálogo do OpenRouter. Nessa comparação, a classificação “Viável” indica que o ecossistema apresenta os componentes necessários para processar a modalidade, diretamente ou por meio da execução sequencial de modelos. Essa classificação representa a viabilidade técnica da composição, não necessariamente seu desempenho efetivo, que ainda deve ser verificado experimentalmente.

Tabela 5 – Viabilidade por ecossistema (composição sequencial)

Critério	**GPT**	**Claude**	**Llama**	**Qwen**	**Gemini**	**Nemo-tron**	**North**
Texto	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso
Texto+Imagem	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso	Falha
Áudio	Sucesso	Falha	Sucesso	Falha	Sucesso	Sucesso *	Falha
Áudio+Imagem	Sucesso	Falha	Sucesso	Falha	Sucesso	Sucesso *	Falha

,

Elaborado pelo próprio autor (2026).

A consideração dos ecossistemas modificou os resultados da comparação, pois permitiu superar algumas das limitações encontradas durante a avaliação isolada. Com o auxílio de outros modelos disponíveis no OpenRouter, os ecossistemas GPT e Llama passaram a contemplar os cenários com áudio, aproximando-se da cobertura oferecida pelo Gemini. O Nemotron também apresentou recursos para compor um fluxo multimodal, embora sua aplicação ainda dependa da validação da interpretação dos conteúdos sonoros. Em contrapartida, a ausência de alternativas para o processamento de áudio manteve Claude e Qwen com cobertura parcial, enquanto o North permaneceu limitado às entradas textuais.

5.5 Análise Individual dos Modelos

Para aprofundar as duas comparações anteriores, cada modelo foi analisado individualmente tanto em relação aos resultados obtidos no *benchmark* quanto às possibilidades oferecidas por seu ecossistema no OpenRouter. Dessa forma, a análise considera primeiro o comportamento do modelo utilizado isoladamente e, em seguida, verifica se suas limitações podem ser contornadas com o uso de modelos auxiliares.

Foram considerados seis critérios: processamento textual, processamento de imagem, processamento de áudio, processamento conjunto de áudio e imagem, geração do

`JSON`

esperado e adequação da recomendação produzida. Na análise dos ecossistemas, também foram observadas a disponibilidade de modelos especializados e a possibilidade de executar múltiplos modelos em sequência.

O processamento de uma modalidade foi considerado bem-sucedido quando a requisição foi aceita pelo provedor e produziu uma resposta associada ao conteúdo enviado. Portanto, o simples fato de a API aceitar um arquivo não significa que seu conteúdo tenha sido corretamente compreendido. Essa diferença é especialmente importante na análise do Nemotron, que aceitou uma das solicitações contendo áudio, mas não identificou adequadamente a ocorrência relatada.

A adequação da resposta foi examinada considerando a correspondência entre a ocorrência, o serviço recomendado, o número informado e o resumo produzido. Esse critério possui natureza qualitativa. A avaliação foi realizada com base nos dados disponíveis no *benchmark* e não representa uma validação oficial de todos os telefones locais apresentados pelos modelos.

5.5.1 GPT-5.5

O GPT-5.5 concluiu os dois cenários que não continham áudio. No cenário textual, referente ao veículo parado no acostamento, recomendou a Polícia Rodoviária Estadual da Bahia e informou o número 198. A resposta considerou corretamente a presença de uma passageira idosa, o anoitecer e a sensação de insegurança. O contexto produzido foi coerente com a entrada e apresentou todos os campos exigidos.

No cenário composto por texto e imagem, o modelo recomendou o Corpo de Bombeiros, por meio do número 193, para a árvore caída sobre fios elétricos. A recomendação foi considerada satisfatória, pois a ocorrência envolvia risco de choque, faíscas e bloqueio da via.

Entretanto, os dois cenários que continham áudio falharam antes da inferência. Em ambos os casos, o OpenRouter retornou o código 404 e a mensagem

`No endpoints found that support input audio`

. Portanto, a falha não ocorreu na interpretação semântica do áudio, mas na indisponibilidade de um *endpoint* compatível com essa modalidade para o modelo e a rota utilizados.

Por não conseguir atender a todos os cenários por meio de um único modelo, o GPT-5.5 foi uma das alternativas que exigiu a utilização de um modelo auxiliar. Para os cenários com áudio, foi utilizado o GPT Audio, responsável por processar o arquivo e produzir uma representação textual do relato. Esse conteúdo foi posteriormente encaminhado ao GPT-5.5, que realizou a classificação da ocorrência e gerou a resposta no formato esperado. No cenário com áudio e imagem, a representação textual produzida pelo GPT Audio foi enviada ao GPT-5.5 juntamente com a imagem.

5.5.2 Claude Opus 4.8

O Claude Opus 4.8 concluiu somente o cenário textual. A resposta recomendou "Polícia Rodoviária / Polícia Militar" e apresentou o número 190. O resumo interpretou adequadamente o veículo parado, a presença da idosa e a insegurança no acostamento. Apesar disso, o nome do serviço ficou ambíguo, pois reuniu dois órgãos diferentes enquanto apresentava apenas o telefone da Polícia Militar.

No cenário de texto e imagem, o provedor retornou o código 400 e a mensagem Provider returned error. O resultado não permite afirmar que o Claude seja incapaz de processar imagens de forma geral, mas demonstra que a solicitação visual não foi processada com sucesso na configuração usada no *benchmark*. Diferentemente de uma mensagem explícita de incompatibilidade, esse erro é genérico e pode estar relacionado ao provedor selecionado pelo OpenRouter, ao formato da requisição ou à disponibilidade momentânea da rota.

Nos dois testes contendo áudio, o resultado foi o código 404 com a mensagem

`No endpoints found that support input audio`

. Dessa forma, não houve processamento do áudio nem da combinação entre áudio e imagem.

Por não concluir o cenário visual na configuração inicial, o Claude Opus 4.8 também foi avaliado considerando os demais modelos de seu ecossistema. A disponibilidade de modelos Claude com suporte a imagens permitiu utilizar um modelo auxiliar para interpretar o conteúdo visual e encaminhar sua descrição textual ao Claude Opus 4.8. Entretanto, não foi identificado no OpenRouter um modelo do ecossistema Claude capaz de processar áudio. Dessa forma, a composição permitiu atender ao cenário de texto e imagem, mas os dois cenários contendo áudio permaneceram sem processamento.

5.5.3 Llama 4 Maverick

O Llama 4 Maverick concluiu os cenários textual e visual. No primeiro, recomendou a Polícia Rodoviária Federal e informou o número 191. A resposta identificou corretamente o risco de permanecer no acostamento com uma pessoa idosa durante o anoitecer. Entretanto, a entrada não indicava que o veículo estava em uma rodovia federal. Dessa forma, a escolha da Policial Rodoviário Federal (PRF) foi considerada parcialmente adequada, pois introduziu uma especificidade que não estava confirmada pelo contexto.

No cenário da árvore sobre a fiação, o modelo recomendou a Coelba e apresentou o número 0800 071 0909. A decisão de acionar a concessionária de energia foi compatível com o risco elétrico descrito e o resumo reconheceu as faíscas e a obstrução da rua. A resposta foi formalmente válida e utilizou as informações da ocorrência.

Os dois cenários com áudio retornaram o código 404 e a mensagem

`No endpoints found that support input audio`

. Assim, o modelo não foi testado semanticamente nessas ocorrências, pois a integração não encontrou um *endpoint* que aceitasse essa modalidade.

Na segunda análise, a limitação relacionada ao áudio foi contornada com a utilização do Muse Spark 1.2, modelo do ecossistema da Meta capaz de receber entradas sonoras e visuais. O modelo auxiliar foi responsável por interpretar o áudio e produzir uma representação textual da ocorrência, posteriormente encaminhada ao Llama 4 Maverick para classificação e geração do

`JSON`

. No cenário com áudio e imagem, ambos os conteúdos foram inicialmente processados pelo Muse Spark antes do envio das informações extraídas ao modelo principal. Dessa forma, a composição permitiu incluir os quatro cenários na avaliação do ecossistema.

5.5.4 Qwen 3.6 Plus

O Qwen 3.6 Plus também concluiu apenas os cenários sem áudio. No cenário textual, retornou "Polícia Militar / Rodoviária" e o número 190. A resposta reconheceu a necessidade de apoio para segurança, sinalização e remoção do veículo. Assim como ocorreu com o Claude, o nome do serviço reuniu órgãos distintos, enquanto o número fornecido correspondia apenas à Polícia Militar. A recomendação foi funcional, mas pouco precisa na identificação institucional.

No cenário de texto e imagem, o modelo recomendou o Corpo de Bombeiros, com o número 193. A resposta identificou corretamente a queda da árvore, o risco de choque e o bloqueio da via, sendo considerada adequada ao cenário.

Nos testes de áudio e de áudio com imagem, o OpenRouter retornou o código 404 com a mensagem

`No endpoints found that support input audio`

. A falha ocorreu, portanto, por incompatibilidade da rota de inferência com a modalidade sonora.

Na segunda análise, não foi identificado no OpenRouter um modelo do ecossistema Qwen capaz de receber áudio e convertê-lo em uma representação textual para o Qwen 3.6 Plus. Embora a família apresente outros modelos com recursos visuais, essa capacidade não soluciona a principal limitação observada, pois o modelo principal já processou satisfatoriamente texto e imagem. Dessa forma, a avaliação do ecossistema não ampliou a cobertura do Qwen, que permaneceu restrito aos dois cenários sem áudio.

5.5.5 Gemini 3.5 Flash

O Gemini 3.5 Flash foi o único modelo que concluiu os quatro cenários e retornou os três campos exigidos em todas as respostas.

No cenário textual, recomendou a Polícia Militar, por meio do número 190, considerando o veículo parado, o anoitecer, a presença da pessoa idosa e a insegurança relatada. A escolha foi considerada coerente por evitar assumir que a via era necessariamente estadual ou federal.

No cenário de texto e imagem, indicou a Neoenergia Coelba e informou o número 116. O resumo identificou a árvore, os fios energizados, as faíscas e a obstrução da via. A resposta demonstrou integração entre o relato textual, a imagem e o contexto geográfico.

No cenário exclusivamente sonoro, o modelo identificou que o áudio relatava um gato atropelado e recomendou um hospital veterinário de Santo Antônio de Jesus. O conteúdo foi semanticamente compatível com a gravação e demonstrou processamento efetivo da fala. Entretanto, como o modelo apresentou um estabelecimento e um telefone locais específicos, essa informação deve ser verificada externamente antes de ser considerada factual. O *benchmark* demonstra que o modelo produziu uma resposta contextualizada, mas não comprova, isoladamente, que o contato esteja atualizado.

No cenário composto por áudio e imagem, recomendou o Corpo de Bombeiros, pelo número 193, e descreveu a entrada de água na residência durante fortes chuvas. A resposta correspondeu às duas modalidades e ao contexto geográfico de Jequié.

Por apresentar suporte nativo às modalidades de texto, imagem e áudio, o Gemini 3.5 Flash não exigiu a utilização de modelos auxiliares na segunda análise. Os quatro cenários puderam ser processados diretamente pelo mesmo modelo, incluindo a combinação de áudio e imagem em uma única requisição. Dessa forma, seu ecossistema não precisou recorrer ao processamento sequencial, mantendo o fluxo de classificação mais simples entre as alternativas avaliadas.

5.5.6 Nemotron 3 Nano Omni

O Nemotron produziu respostas nos três primeiros cenários, mas apresentou problemas importantes de interpretação.

No cenário textual, recomendou a Polícia Rodoviária Federal e informou o número 191. O resumo reconheceu o veículo parado, o período noturno e a presença de uma pessoa idosa. Contudo, assim como no Llama, a escolha da PRF pressupôs que a ocorrência estivesse em rodovia federal, informação não fornecida no cenário. Por isso, a recomendação foi considerada apenas parcialmente adequada.

No cenário com texto e imagem, recomendou “Neoenergia” e apresentou o telefone 0800 01 5 5 200. Embora tenha reconhecido corretamente o risco elétrico, o número foi retornado com formatação incomum e não pôde ser considerado confiável apenas com base no *benchmark*. O nome genérico da empresa também não especificou claramente a concessionária responsável pela região.

No cenário de áudio, a requisição foi tecnicamente aceita, diferentemente do que ocorreu com a maioria dos modelos. Entretanto, a resposta indicou a Polícia Militar, pelo número 190, e afirmou que a ocorrência não havia sido especificada. O áudio relatava um gato atropelado. Portanto, o modelo demonstrou compatibilidade técnica

com o arquivo sonoro, mas não conseguiu compreender seu conteúdo. Esse resultado deve ser classificado como falha semântica, não como sucesso do cenário.

No cenário com áudio e imagem, o registro ficou vazio: não houve serviço, telefone, contexto, código de erro ou métricas. Isso indica que a execução não produziu um resultado aproveitável, mas o arquivo

`CSV`

não permite determinar se houve interrupção da chamada, falha na gravação do resultado ou retorno vazio do provedor.

Na segunda análise, o Nemotron 3 Nano Omni deixou de ser utilizado como único responsável por interpretar as entradas e classificar a ocorrência. O modelo passou a atuar como componente de percepção, encarregado de extrair as informações do áudio e da imagem, enquanto o conteúdo textual resultante foi encaminhado a outro modelo Nemotron para classificação e geração do

`JSON`

. Essa separação buscou reduzir a quantidade de tarefas atribuídas a uma única execução. Entretanto, como o problema observado estava justamente na compreensão do áudio, a eficácia da composição permaneceu condicionada à qualidade da interpretação produzida pelo Nano Omni.

5.5.7 North Mini Code

O North Mini Code concluiu apenas o cenário textual. A resposta recomendou a Polícia Militar, informou o número 190 e resumiu adequadamente o veículo parado, o anoitecer, a presença da passageira idosa e a insegurança. Embora simples, a resposta cumpriu a estrutura

`JSON`

exigida e foi considerada satisfatória para essa entrada.

No cenário com imagem, o OpenRouter retornou o código 404 e a mensagem

`No endpoints found that support image input`

. Essa mensagem demonstra explicitamente que nenhum endpoint disponível para aquela rota aceitava conteúdo visual.

O cenário exclusivamente sonoro retornou o código 404 com

`No endpoints found that support input audio`

. Já o cenário contendo áudio e imagem voltou a apresentar incompatibilidade com imagem. Dessa forma, o modelo demonstrou suporte apenas à entrada textual na integração utilizada.

O North apresentou baixo custo e tempo de resposta relativamente curto na única execução concluída. Entretanto, sua natureza voltada principalmente para tarefas de código e sua ausência de compatibilidade multimodal tornaram o modelo inadequado aos requisitos centrais da aplicação.

Na segunda análise, não foram identificados no OpenRouter modelos do ecossistema Cohere capazes de processar imagens ou áudios para complementar o North Mini Code. Por isso, não foi possível construir um fluxo com modelos auxiliares que convertesse essas entradas em conteúdo textual. A avaliação do ecossistema não ampliou sua cobertura, que permaneceu restrita ao cenário composto exclusivamente por texto.

5.6 Conclusão do Benchmarking

O desenvolvimento do Centro de Ajuda também ampliou a compreensão sobre o funcionamento de modelos multimodais em aplicações integradas por API. Durante a implementação e a realização dos testes, foi possível identificar que a multimodalidade apresentada nas interfaces de conversação dos diferentes ecossistemas nem sempre corresponde às capacidades disponíveis diretamente em seus modelos ou *endpoints*.

Em alguns casos, as interfaces disponibilizadas pelos desenvolvedores combinam diferentes modelos e ferramentas para receber texto, áudio e imagem. Essa composição pode transmitir a impressão de que um único modelo processa todas as modalidades,

embora parte do conteúdo seja tratada previamente por componentes especializados. Quando o modelo é acessado isoladamente por API, essas funcionalidades podem não estar disponíveis na mesma requisição.

Essa diferença foi observada no *benchmark* realizado por meio do OpenRouter. Entre os modelos avaliados isoladamente, apenas o Gemini 3.5 Flash processou diretamente texto, imagem e áudio nos quatro cenários. Os demais apresentaram diferentes níveis de compatibilidade, variando entre o suporte a texto e imagem, a limitação exclusiva ao texto e a aceitação técnica de arquivos sem a compreensão adequada de seu conteúdo.

A análise dos ecossistemas demonstrou que algumas dessas limitações podem ser contornadas pela execução sequencial de modelos especializados. Nesse tipo de fluxo, um modelo pode interpretar determinada modalidade e encaminhar o conteúdo extraído ao modelo responsável pela classificação final. Portanto, a adoção de inteligência artificial em aplicações multimodais exige a verificação das capacidades efetivamente disponibilizadas pela API, não sendo suficiente considerar apenas os recursos presentes nas interfaces de chat ou descritos de maneira geral para cada ecossistema.

6 Conclusão

O Centro de Ajuda demonstrou viabilidade técnica como apoio ao direcionamento emergencial, integrando entrada multimodal, geocodificação, *prompts* e OpenRouter para entregar

`service_name`

/

`phone_number`

/

`emergency_context`

com redirecionamento

`tel:`

.

6.1 Avaliação dos Modelos

A avaliação realizada por meio do *benchmark* permitiu observar diferenças importantes entre os modelos quanto à compatibilidade com texto, áudio e imagem, à geração de respostas estruturadas, à adequação das recomendações, ao tempo de processamento e ao custo. Entre os modelos avaliados, apenas o Gemini 3.5 Flash processou diretamente as entradas de texto, imagem e áudio nos quatro cenários propostos. Os demais apresentaram limitações de compatibilidade com uma ou mais modalidades, exigindo, para uma eventual utilização no sistema, o emprego de modelos especializados ou de etapas adicionais de processamento. Os testes também demonstraram que a disponibilidade de uma modalidade depende não apenas das capacidades anunciadas para cada família, mas dos modelos e *endpoints* efetivamente acessíveis durante a integração. A análise dos ecossistemas complementou essa avaliação ao considerar a possibilidade de utilizar modelos especializados em diferentes etapas do processamento.

6.2 Propostas Futuras

Embora os resultados obtidos tenham demonstrado a viabilidade da proposta, o Centro de Ajuda ainda possui possibilidades de aprimoramento antes de uma eventual utilização em situações reais. As propostas apresentadas a seguir envolvem a evolução técnica da aplicação, a ampliação do processo de avaliação, a investigação de outros modelos de linguagem e o desenvolvimento de mecanismos capazes de aumentar a confiabilidade das recomendações produzidas.

6.2.1 Aperfeiçoamento da aplicação

Como continuidade deste trabalho, a prova de conceito pode ser ampliada por meio da implementação completa da aplicação cliente e do aperfeiçoamento de aspectos relacionados à segurança, acessibilidade, desempenho e tratamento de falhas. Entre as possíveis melhorias estão a adoção de mecanismos de proteção dos dados enviados, a ampliação da compatibilidade com diferentes dispositivos e a inclusão de recursos de acessibilidade, como leitura de tela, comandos por voz, textos com maior contraste e formas alternativas de interação. Também podem ser desenvolvidos recursos para funcionamento parcial em situações de conexão instável ou indisponível. Entre eles, destacam-se o armazenamento local de números oficiais, a apresentação de orientações básicas previamente cadastradas e a manutenção temporária das informações fornecidas pelo usuário até que a conexão seja restabelecida. Entretanto, qualquer armazenamento local de relatos, imagens, áudios ou dados de localização deverá considerar mecanismos de proteção e descarte seguro dessas informações. No servidor, podem ser implementadas estratégias de repetição automática das requisições, definição de limites de tempo de resposta e seleção de modelos alternativos quando o serviço principal estiver indisponível. Também seria possível utilizar mecanismos de monitoramento para registrar falhas, identificar períodos de instabilidade e acompanhar o desempenho dos provedores utilizados pela aplicação.

6.2.2 Ampliação dos cenários de avaliação

A avaliação pode ser expandida com uma quantidade maior de cenários, abrangendo diferentes tipos de ocorrência, níveis de gravidade, localidades e serviços responsáveis pelo atendimento. Novos experimentos poderiam contemplar situações médicas, acidentes de trânsito, incêndios, desastres naturais, ocorrências policiais, violações de direitos e problemas relacionados a serviços públicos ou concessionárias. Também podem ser incluídos relatos ambíguos, incompletos ou escritos com erros, além de áudios com ruídos, falas interrompidas, diferentes sotaques e baixa qualidade de gravação. No caso das imagens, seria relevante utilizar registros com pouca iluminação, baixa resolução, enquadramento inadequado ou elementos parcialmente ocultos. Outros cenários poderiam apresentar informações complementares ou contraditórias entre texto, áudio e imagem, permitindo analisar como cada modelo identifica e resolve possíveis divergências. A repetição de cada cenário permitiria calcular médias e medidas de variação mais representativas para custo, tempo de resposta, consumo de *tokens* e estabilidade. Dessa forma, seria possível reduzir a influência das condições específicas de uma única execução e produzir comparações mais consistentes entre os modelos avaliados. A realização dos testes em diferentes horários e períodos também poderia contribuir para identificar variações relacionadas à disponibilidade e à carga dos serviços em nuvem.

6.2.3 Avaliação de outros modelos

Trabalhos futuros também podem avaliar modelos menores ou executados localmente, como os pertencentes às famílias Llama, Gemma e Qwen. Essa abordagem permitiria investigar alternativas com menor custo de operação, maior controle sobre os dados processados e menor dependência de plataformas externas. A execução local também poderia ser útil em ambientes com restrições de conectividade ou com requisitos mais rigorosos de privacidade. Entretanto, essa possibilidade exige a análise dos recursos

computacionais necessários, do tempo de processamento e da capacidade de cada modelo para interpretar texto, áudio e imagem. Em alguns casos, pode ser necessário utilizar modelos especializados em conjunto, como um sistema para transcrição de áudio, outro para análise de imagens e um modelo de linguagem responsável pela classificação final da ocorrência. Além disso, recomenda-se repetir o *benchmark* utilizando os modelos mais modernos disponíveis no período em que as melhorias foram realizadas, considerando a rápida evolução das capacidades multimodais e das interfaces oferecidas pelos provedores. Essas avaliações deverão manter os mesmos cenários, critérios e estrutura de entrada sempre que possível, permitindo comparar os novos resultados com aqueles obtidos neste trabalho.

6.2.4 Validação das recomendações

Os resultados podem ser comparados com classificações de referência previamente definidas e avaliadas por profissionais com conhecimento nas áreas envolvidas. Essa validação permitiria verificar de maneira mais objetiva se o serviço recomendado, o número informado e o contexto apresentado corresponde adequadamente à ocorrência descrita. A participação de profissionais ligados à saúde, segurança pública, defesa civil, assistência social e serviços municipais também poderia auxiliar na elaboração dos cenários e na definição dos critérios de avaliação. Além da identificação do órgão mais adequado, poderiam ser analisados aspectos como gravidade, urgência, clareza da justificativa e presença de recomendações potencialmente inadequadas ou perigosas. A utilização contínua do sistema também poderia contribuir para a formação de uma base de dados com os números e órgãos identificados durante as solicitações, funcionando como um mecanismo de cache organizado por localização, área de atendimento e tipo de ocorrência. Quando um contato já estivesse armazenado e validado para determinado contexto, o sistema poderia utilizá-lo em solicitações posteriores, reduzindo a necessidade de novas pesquisas e a dependência das informações produzidas pelos modelos.

Nesse funcionamento, o modelo seria empregado principalmente na interpretação e na classificação do relato, enquanto os dados de contato seriam recuperados da base. Para evitar a propagação de informações incorretas, a inclusão e a reutilização dos contatos deveriam ser acompanhadas por processos de validação em fontes oficiais, controle de validade e rotinas periódicas de verificação e atualização.

6.3 Considerações Finais

Dessa forma, os resultados obtidos indicam que o Centro de Ajuda é tecnicamente viável como uma ferramenta de apoio ao direcionamento de solicitações emergenciais. A avaliação demonstrou que os modelos de inteligência artificial podem interpretar as informações fornecidas e contribuir para a classificação e o direcionamento das ocorrências, embora sua utilização dependa da compatibilidade das interfaces, da validação dos contatos recomendados e da adoção de mecanismos para tratar erros e indisponibilidades. A prova de conceito conseguiu demonstrar a integração entre a interface, o servidor, os serviços de localização e os modelos de inteligência artificial, atendendo ao objetivo estabelecido para o trabalho. Entretanto, a solução não deve ser compreendida como substituta das centrais oficiais ou da avaliação humana, mas como um recurso intermediário destinado a reduzir dúvidas e facilitar o acesso ao serviço considerado mais adequado. Os resultados alcançados representam, portanto, uma etapa inicial do desenvolvimento da proposta. As possibilidades apresentadas para sua continuidade demonstram que o Centro de Ajuda ainda possui um amplo campo de evolução, tanto em sua implementação quanto na avaliação e na confiabilidade das recomendações. Assim, este trabalho estabelece uma base para novos estudos e aperfeiçoamentos que poderão aproximar a proposta de uma aplicação mais segura, abrangente e adequada a situações reais.

REFERÊNCIAS

ANTHROPIC, . *Claude Opus 4.8 Model Documentation*. Anthropic Docs, 2026. Disponível em: <https://docs.anthropic.com/en/docs/about-claude/models>. Acesso em: 10 jun 2026. Citado 2 vezes nas páginas 34 e 35.

ASSOCIATION, E. E. N. *What is 112*. EENA, 2024. Disponível em: <https://eena.org/about-112/whats-112-all-about/>. Acesso em: 23 jan 2026. Citado na página 18.

ASSOCIATION, N. E. N. *911 Overview and Facts*. NENA, 2024. Disponível em: <https://www.nena.org/page/911overviewfacts>. Acesso em: 23 jan 2026. Citado na página 18.

BROWN, T.; MANN, B.; RYDER, N.; SUBBIAH, M.; KAPLAN, J.; DHARIWAL, P.; NEELAKANTAN, A.; SHYAM, P.; SASTRY, G.; ASKELL, A.; OTHERS, . *Language Models are Few-Shot Learners*. arXiv:2005.14165, 2020. Disponível em: <https://arxiv.org/abs/2005.14165>. Acesso em: 12 feb 2026. Citado 2 vezes nas páginas 23 e 24.

CLOUD, A. *Qwen API Platform*. Qwen, 2026. Disponível em: <https://qwen.ai/apiplatform>. Acesso em: 09 jun 2026. Citado 2 vezes nas páginas 36 e 37.

CLOUD, A. *Qwen3-VL*. GitHub, 2026. Disponível em: <https://github.com/QwenLM/Qwen3-VL>. Acesso em: 09 jun 2026. Citado na página 37.

COHERE, . *North Mini Code 1.0*. Cohere Docs, 2026. Disponível em: <https://docs.cohere.com/docs/north-mini-code-1.0>. Acesso em: 10 jun 2026. Citado 2 vezes nas páginas 39 e 40.

DOSOVITSKIY, A.; BEYER, L.; KOLESNIKOV, A.; WEISSENBORN, D.; ZHAI, X.; UNTERTHINER, T.; DEHGHANI, M.; MINDERER, M.; HEIGOLD, G.; GELLY, S.; OTHERS, . *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. arXiv:2010.11929, 2021. Disponível em: <https://arxiv.org/abs/2010.11929>. Acesso em: 20 feb 2026. Citado na página 27.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. Cambridge: MIT Press, 2016. Citado 4 vezes nas páginas 21, 22, 27 e 29.

GOOGLE, . *Gemini 3.5 Flash Model Documentation*. Google AI for Developers, 2026. Disponível em: <https://ai.google.dev/gemini-api/docs/models>. Acesso em: 10 jun 2026. Citado 2 vezes nas páginas 37 e 38.

GOOGLE, . *Google AI Platform*. Google AI, 2026. Disponível em: <https://ai.google.dev/>. Acesso em: 10 jun 2026. Citado na página 38.

IBM, . *What is Machine Learning*. IBM Topics, 2024. Disponível em: <https://www.ibm.com/topics/machine-learning>. Acesso em: 03 feb 2026. Citado na página 20.

META, . *Llama 4 Maverick*. Meta AI, 2026. Disponível em: <https://www.llama.com/models/llama-4/>. Acesso em: 08 jun 2026. Citado 2 vezes nas páginas 35 e 36.

META, . *Llama API Overview*. Meta Developer Docs, 2026. Disponível em: <https://llama.developer.meta.com/docs/overview/>. Acesso em: 08 jun 2026. Citado na página 36.

MITCHELL, T. *Machine Learning*. New York: McGraw-Hill, 1997. Citado na página 19.

MOSS, E. *History of Emergency Numbers*. Journal of Social History, 2017. Disponível em: <https://doi.org/10.1093/jsh/shx101>. Acesso em: 12 jan 2026. Citado 2 vezes nas páginas 13 e 14.

NVIDIA, . *Nemotron 3 Nano Omni 30B A3B Reasoning*. NVIDIA Build, 2026. Disponível em: <https://build.nvidia.com/nvidia/nemotron-3-nano-omni-30b-a3b-reasoning>. Acesso em: 10 jun 2026. Citado 2 vezes nas páginas 38 e 39.

OPENAI, . *GPT-4 Technical Report*. arXiv:2303.08774, 2023. Disponível em: <https://arxiv.org/abs/2303.08774>. Acesso em: 18 feb 2026. Citado 4 vezes nas páginas 25, 26, 27 e 29.

OPENAI, . *Prompt Engineering Guide*. OpenAI Platform Docs, 2024. Disponível em: <https://platform.openai.com/docs/guides/prompt-engineering>. Acesso em: 05 mar 2026. Citado 2 vezes nas páginas 24 e 25.

OPENAI, . *GPT-5.5 Model Documentation*. OpenAI Developer Platform, 2026. Disponível em: <https://developers.openai.com/api/docs/models/gpt-5.5>. Acesso em: 10 jun 2026. Citado 2 vezes nas páginas 33 e 34.

RADFORD, A.; KIM, J. W.; XU, T.; BROCKMAN, G.; MCLEAVEY, C.; SUTSKEVER, I. *Robust Speech Recognition via Large-Scale Weak Supervision*. arXiv:2212.04356, 2022. Disponível em: <https://arxiv.org/abs/2212.04356>. Acesso em: 24 feb 2026. Citado na página 28.

RAPIDSOS, . *Unite Platform for Public Safety*. RapidSOS, 2025. Disponível em: <https://wordpress-1533060-5913121.cloudwaysapps.com/public-safety/unite/>. Acesso em: 21 jan 2026. Citado na página 17.

RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. Berkeley: Pearson, 2020. Citado na página 19.

TELECOMUNICAÇÕES, A. N. D. *Numeração de Serviços de Emergência*. ANATEL, 2025. Disponível em: <https://www.gov.br/anatel/pt-br/regulado/numeracao>. Acesso em: 14 jan 2026. Citado na página 14.

UNIJUÍ, R. C. \. S. -. *Conhecimento da População sobre Números de Emergência*. Revista Contexto \& Saúde, 2024. Disponível em: <https://www.revistas.unijui.edu.br/index.php/contextoesaude/article/view/14835>. Acesso em: 16 jan 2026. Citado na página 14.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, L.; POLOSUKHIN, I. *Attention Is All You Need*. arXiv:1706.03762,

2017. Disponível em: <https://arxiv.org/abs/1706.03762>. Acesso em: 09 feb 2026. Citado 3 vezes nas páginas 22, 23 e 30.

WEI, J.; WANG, X.; SCHUURMANS, D.; BOSMA, M.; ICHTER, B.; XIA, F.; CHI, E.; LE, Q.; ZHOU, D. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. arXiv:2201.11903, 2022. Disponível em: <https://arxiv.org/abs/2201.11903>. Acesso em: 09 mar 2026. Citado na página 25.